

Academic year
2025 - 2026

A Study of Weakly Supervised Instance Segmentation Methods

Stein Vandenbroeke

Master's thesis

Supervisors

dr. Arian Sabaghi K., UAntwerpen

dr. prof. José Oramas M., UAntwerpen



University of Antwerp
| Faculty of Science

Disclaimer Master's thesis

This document is an examination document that has not been corrected for any errors identified. Without prior written permission of both the supervisor(s) and the author(s), any copying, copying, using or realizing this publication or parts thereof is prohibited. For requests for information regarding the copying and/or use and/or realisation of parts of this publication, please contact to the university at which the author is registered.

Prior written permission from the supervisor(s) is also required for the use for industrial or commercial utility of the (original) methods, products, circuits and programs described in this thesis, and for the submission of this publication for participation in scientific prizes or competitions.

This document is in accordance with the faculty regulations related to this examination document and the Code of Conduct. The text has been reviewed by the supervisor and the attendant.

Abstract. Training deep neural networks for instance segmentation requires expensive, time-consuming pixel-level annotations. Weakly Supervised Instance Segmentation (WSIS) solves this problem by training models using only image-level labels. However, properly evaluating these models’ ability to separate distinct, interacting objects remains a challenge. This thesis investigates the current state of WSIS by comparatively evaluating methods—including Background Activation Suppression (BAS), Beyond Semantic to Instance Segmentation (BESTIE), and Complete Instances Mining (CIM)—on the Pascal VOC 2012 and MS COCO datasets. To provide a more accurate assessment of true instance separation, this study introduces a border-pixel occlusion metric that explicitly isolates and measures model performance on overlapping, same-class instances. Furthermore, we examine the performance of WSIS methods on a Weakly Supervised Object Localisation (WSOL) task using the single-instance CUB-200-2011 dataset. Our findings demonstrate that Complete Instances Mining (CIM) significantly outperforms the other evaluated methods, particularly in the challenging task of separating occluded instances of the same class. The occlusion-based evaluation reveals that relying solely on standard Mean Average Precision (mAP) scores across an entire dataset can obscure algorithmic weaknesses in splitting overlapping instances, highlighting the necessity for targeted occlusion metrics and visual mask verification, or for datasets with a large number of instances per image. Additionally, applying the BESTIE WSIS architecture to a simple WSOL task led to a notable degradation in localisation performance compared to the baseline WSOL approach.

AI disclaimer. AI tools such as Google Gemini were used in the creation of this work as supporting tools. Grammarly AI functionality was used to rephrase sentences and to do spelling checks in the report.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	From WSOL to WSIS: A Shift in Focus	1
1.3	Problem Statement and Research Questions	2
1.4	Contributions and Thesis Outline	2
2	Background	3
2.1	Convolutional Neural Networks (CNN)	3
2.2	Types of Outputs for Neural Networks	3
2.2.1	Classification Neural Network	5
2.2.2	Regression Neural Network	6
2.2.3	Segmentation Neural Network	6
2.3	Explainable AI (XAI)	7
2.3.1	Input Modification Methods	7
2.3.2	Deconvolutional Methods	8
2.3.3	Input Reconstruction Methods	8
2.4	Localisation and Segmentation	8
2.5	Landscape of Image Supervision Techniques	9
2.5.1	Output types	9
2.5.2	Amount of Supervision	9
2.5.3	WSOL, WSSS, WSIS	10
2.6	Dataset Annotation	10
2.6.1	Instance Pixel Mask Annotations	10
2.6.2	Image-Level Labels Annotations	11
2.6.3	Point-Level Annotations	11
3	Methodology	13
3.1	Overview	13
3.1.1	Comparing and Testing the Methods	13
3.1.2	Testing whether the WSIS approach has an impact on a WSOL task	13
3.2	WSOL methods	15
3.2.1	Background Activation Suppression (BAS)	15
3.3	WSSS methods	17
3.3.1	Background Activation Suppression extended (BAS WSSS)	17
3.4	WSIS methods	17
3.4.1	Background Activation Suppression extended modified (BAS WSIS)	17
3.4.2	Beyond Semantic to Instance Segmentation (BESTIE)	18
3.4.3	Complete Instances Mining (CIM)	19
4	Experiments and results	21
4.1	Metrics	21
4.1.1	Intersection over Union	21
4.1.2	Mean Average Precision	21
4.1.3	Occlusion Metric	22

4.2	Datasets	26
4.2.1	CUB-200-2011	26
4.2.2	VOC 2012	26
4.2.3	Data Challenges	29
4.3	Analyse mAP score w.r.t Occlusion and instance counts	31
4.3.1	BAS (Background Activation Suppression)	31
4.3.2	BESTIE (Beyond Semantic to Instance Segmentation)	36
4.3.3	CIM (Complete Instance Mining)	41
4.3.4	Comparing the Methods	41
4.3.5	Performance of WSIS Method on Single Instance Dataset	46
5	Conclusion	49
5.1	Answers to Research Questions	49
5.1.1	The Current State of WSIS	49
5.1.2	WSOL Metrics for WSIS	49
5.1.3	Instance Segmentation for WSOL	49
5.2	Limitations of the Work	50
5.3	Future Work	50
5.3.1	Evaluating Additional WSIS Architectures	50
5.3.2	Closing the Gap with Supervised Learning	50
5.3.3	Completing the BESTIE COCO Evaluation	50
5.3.4	Expanding the WSOL Compatibility Analysis	50

1 Introduction

1.1 Background and Motivation

Training an image localisation or segmentation DNN (Deep Neural Network) requires a large annotated dataset. However, creating fully annotated datasets for complex or large tasks requires substantial manual effort. For example, annotating a pixel mask for an image with a single instance takes, on average, ± 79.2 s (See Section 2.6.1). Because datasets consist of thousands of images, the time and cost of labelling everything can be substantial. When using a manual labour platform like, for example, AWS Mechanical Turk [3] with a recommended pay of 0.20 per minute, a small dataset of only $\pm 10,000$ single-instance images already costs \$2,640. Imagine doing this for self-driving car applications, where every instance in the image must be annotated and vastly more training data is required.

Even with these costs, impressive supervised instance localisation models exist, such as Meta's SAM (Segment Anything Model) [34][7], which are trained using large manually annotated datasets. Furthermore, while there are good, freely available annotated image datasets such as MS COCO [26] and Pascal VOC 2012 [14], they do not cover all possible tasks. Alternatively, one could rely on Meta's open-weight SAM models; however, these come with restrictive licensing, do not work for every imaginable use case, and are computationally heavy to run because they are designed to segment "anything."

Therefore, less labour-intensive methods using Weakly Supervised Learning were created, offering exactly this solution. It drastically reduces the required annotation work, making it highly relevant for specific use cases or scaling applications like self-driving cars [2], image editing [2], surveillance [24], and object counting [19]. As there is already a large amount of research into Weakly Supervised Object Localisation (Finding the location of an object in an image), we started with a simple question: "Is there a way to detect multiple objects when we use weakly supervised multiple object localisation?"

1.2 From WSOL to WSIS: A Shift in Focus

With these research questions in place, we began exploring the field. After reading many papers and searching for existing research using queries like "Weakly-supervised multiple object localisation," it seemed there were currently no papers that effectively solved this problem. Thus, initial efforts focused on finding independent solutions. This process led to the realisation that using masks rather than bounding boxes was necessary because the exact number of instances was unknown. This realisation opened up a new branch of research. Instead of searching for "Weakly-supervised multiple object localisation," it became clear that there were way more relevant results for Weakly-supervised Semantic Segmentation. This eventually led to Weakly-supervised Instance Segmentation. A relatively small research subfield whose goal perfectly matched the initial objective: to split multiple instances on an image of the same class using Weakly Supervised Learning. This breakthrough came relatively late in the research process because of the small number of papers on this topic and a focus on the wrong search terms (e.g., multiple object localisation rather than Instance Segmentation). After the discovery, the following question came up: "Is there still a certain flaw that can be solved?" Because none of the papers clearly identified a flaw in their methods, the research pivoted to analysing existing methods rather than developing a new one.

1.3 Problem Statement and Research Questions

With the knowledge of the already existing WSIS (Weakly-supervised Instance Segmentation) methods. We changed the research question from creating a method to exploring existing methods, then testing them against each other using the same evaluation code. In this evaluation code, we added an additional test dimension in which performance is evaluated based on instance overlap. And finally, to bring the research back to WSOL (Weakly Supervised Object Localisation), we applied a WSIS method in a WSOL use case. This can be summarised into three clear research questions:

- What is the current state of weakly supervised instance segmentations (WSIS)?
- Can the metrics of weakly supervised object localisation be used for measuring weakly supervised instance localisation (WSIS)?
- Can we use instance segmentation for weakly supervised object localisation (WSOL)?

1.4 Contributions and Thesis Outline

To answer these questions, this research introduces a few key contributions. Within the evaluation code, an additional test dimension was added to evaluate model performance specifically by the degree to which items overlap. Finally, to bring the research back to its initial roots, a WSIS method was applied in a WSOL use case.

This thesis is structured as follows: Chapter 2 (Background) contains the necessary background information to understand this thesis; in Chapter 3 (Methodology), we explain what we will do and how each method works. Chapter 4 (Experiments and Results) goes over the dataset, metrics, and specific challenges of the experiments and results. Finally, in Chapter 5 (Conclusion), we give a concise overview of our findings and possible future work.

2 Background

2.1 Convolutional Neural Networks (CNN)

A Convolutional Neural Network is a type of deep neural network that enables grouping of neighbouring input values (to find structures). Common use cases include image classification and image segmentation. It works as follows: for an RGB input image, there are 3 channels: R (red), G (green), and B (blue). Let's say a value between 0 and 255 represents each colour. If we have an RGB image of size 228×228 , we have a total of $228 \times 228 \times 3 = 155952$ input neurons. Creating a normal, fully connected neural net would cause the number of parameters to grow exponentially with the number of layers, making our network way too big to train or use in practice. Thus, we use a Convolutional Neural Network that contains the following layers:

- **Input Layer:** This is the raw input data. Most of the time, an image is a 3D volume. $height \times width \times depth$ (with depth the colour)
- **Convolutional Layer:** This is the most important layer of the CSS. Here, the kernel is dot-producted with a part of the image of the same size as the kernel. This allows us to extract important features of the image (such as lines). See figure (2.1)
- **Activation Layer:** This is the basic non-linear layer we already know from other deep neural networks, which allows us to learn complex patterns beyond linear relationships (e.g., a ReLU layer).
- **Pooling Layer:** Another important part of the CNN. Here, we will reduce the size of the next layer by pooling the data, for Example, using max pooling or average pooling.
- **Flattening:** This flattens the multi-dimensional feature maps into a one-dimensional vector.
- **Fully Connected Layer:** Here, we have a dense layer for high-level reasoning.
- **Output Layer:** The output layer converts the final scores into probabilities using activation functions such as Sigmoid (binary classification) or Softmax (multi-class classification).

We first have the input layer, then multiple combinations of multiple Convolutional Layer - Activation Layer - Pooling Layer, followed by Flattening, then multiple Fully Connected Layers, and finally the output layer (An example CNN can be seen in 2.3).

2.2 Types of Outputs for Neural Networks

Depending on the specific computer vision task at hand, the architecture and the final output layer of a deep neural network must be adapted. The three most common output paradigms in this domain are classification, regression, and segmentation.

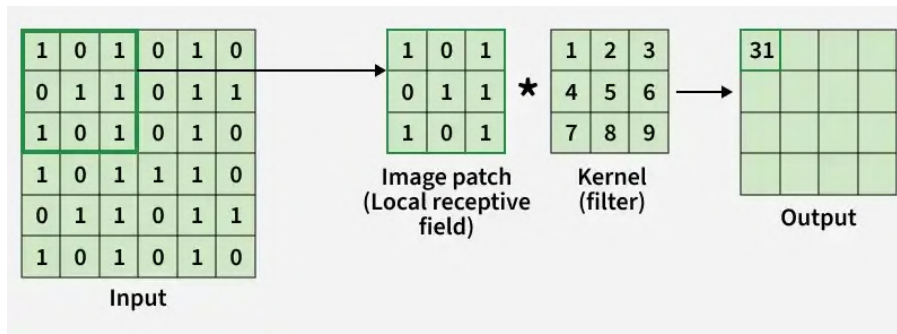


Figure 2.1: The convolution step of a CNN explained with a kernel size of 9; the kernel values (values 1 to 9) themselves will be learned during training.

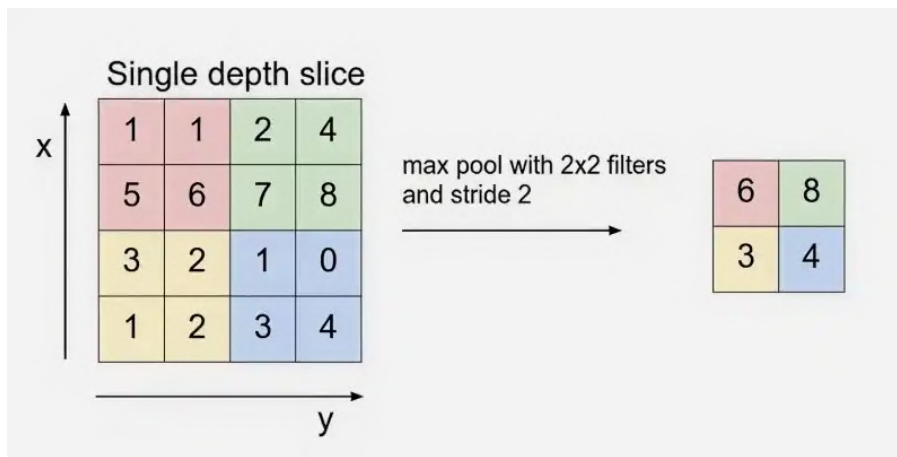


Figure 2.2: Max pooling

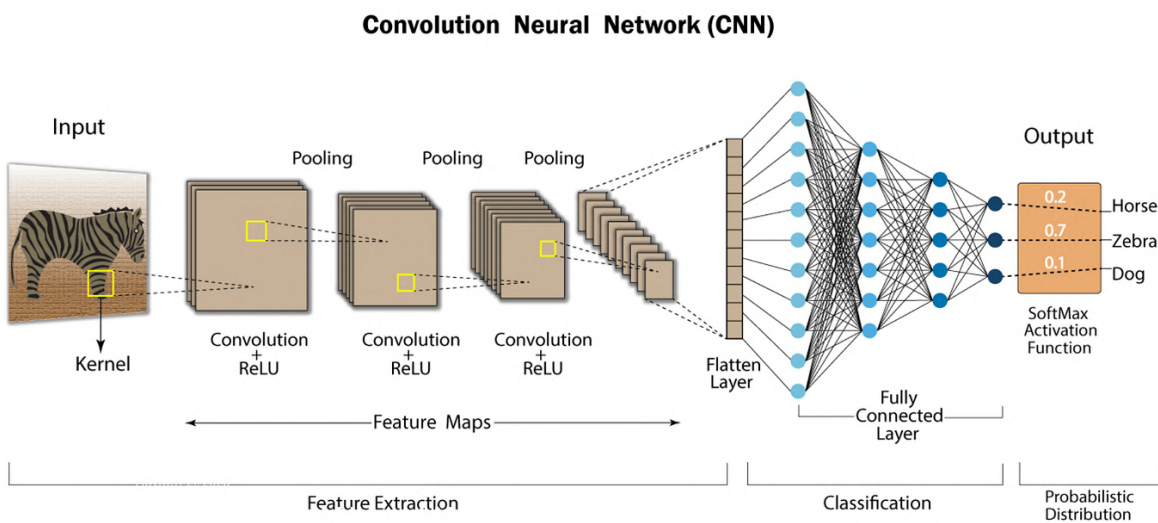


Figure 2.3: Working of a CNN Figure from [4]

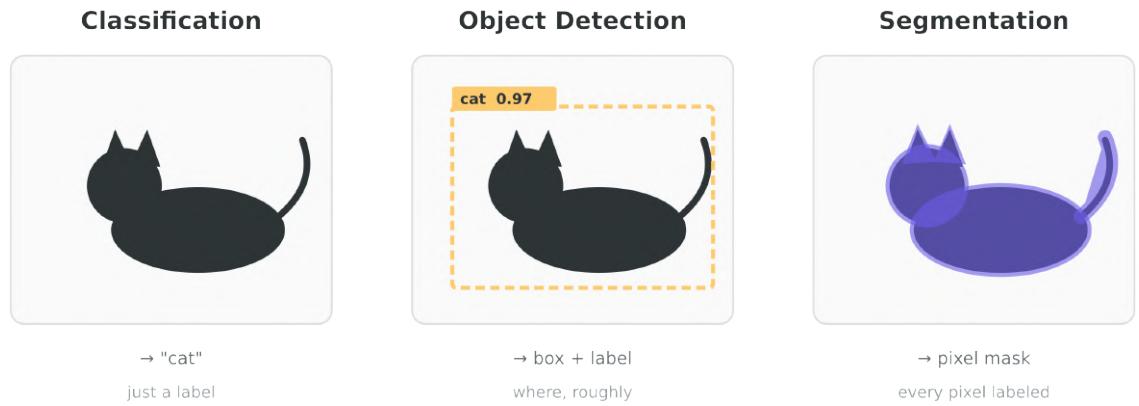


Figure 2.4: Image showing the difference between classification, object detection (example of a regression DNN), and segmentation (Output segmentation DNN). Figure from [1]

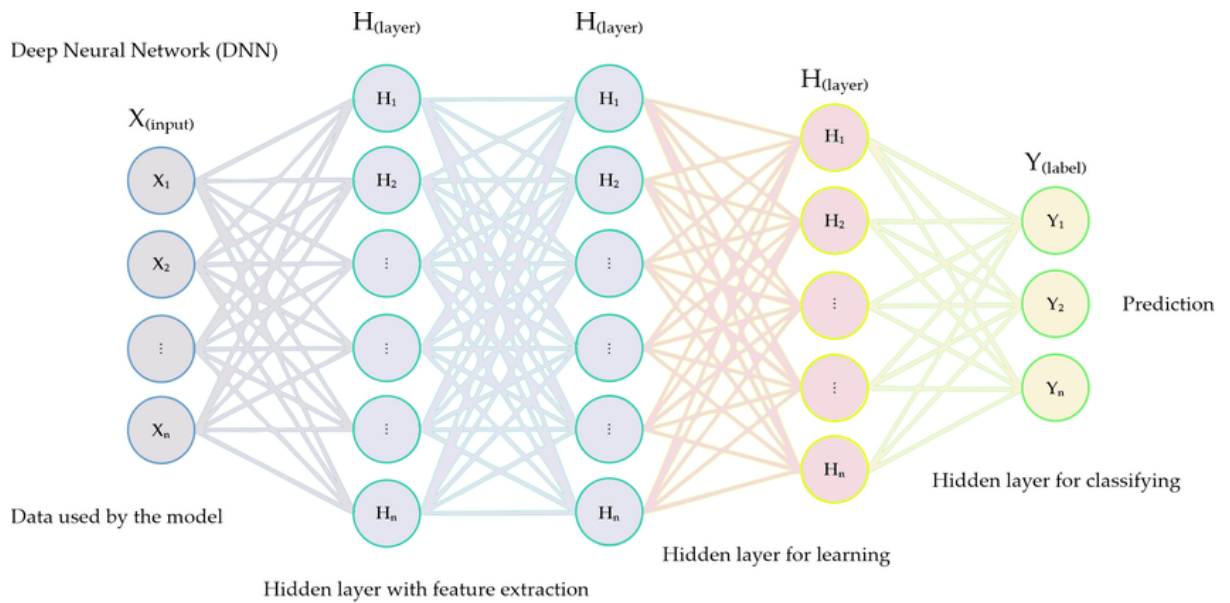


Figure 2.5: Image showing the structure of a classification DNN. Figure from [31]

2.2.1 Classification Neural Network

In an image classification task, the goal is to map an input image to a specific predefined category (The left example in Figure 2.4). In a setting with C potential classes, the network processes the image through a series of convolutional layers (often referred to as the backbone, such as ResNet or VGG) followed by a series of fully connected layers. The final output layer consists of C neurons, each corresponding to a specific class (A simplified DNN example can be seen in Figure 2.5).

Instead of outputting raw numerical values (logits), this final layer usually uses a Softmax activation function. This function normalises the outputs into a probability distribution that sums to one, representing the network's confidence for each class. The class assigned to the image is simply the one corresponding to the output neuron with the highest probability.

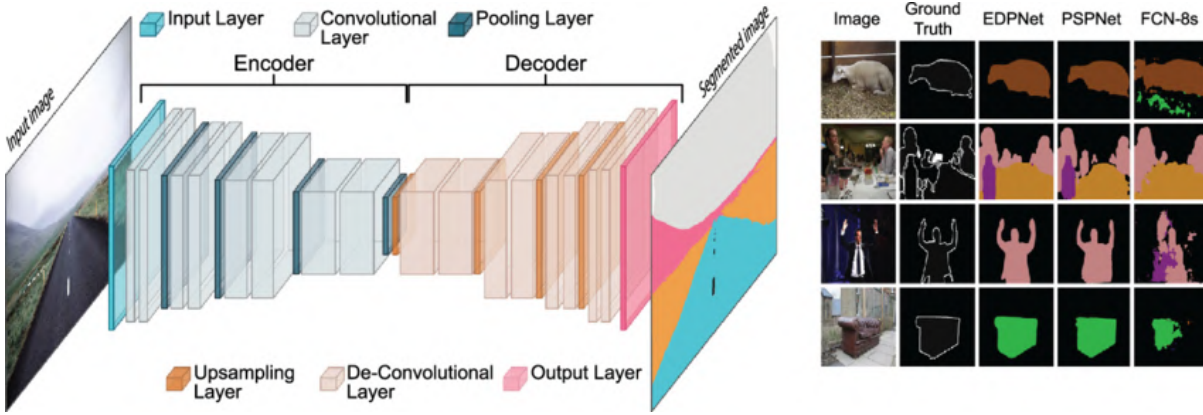


Figure 2.6: Example of a Segmentation Neural Network, where the input is an image and the output is a segmentation image. Image from [8]

2.2.2 Regression Neural Network

While classification deals with discrete, finite categories, regression is used when the network needs to predict continuous numerical values. Rather than a probability distribution, the network outputs direct estimates. A common example is predicting the exact steering angle required for self-driving cars.

In the context of object localisation and detection, regression is vital for predicting bounding boxes. To locate an object, the network's final layer might be designed to output $X = 4$ continuous values, representing the coordinates of the bounding box: $(x_{min}, y_{min}, x_{max}, y_{max})$ (see the middle example of Figure 2.4). Because these values can range over the image resolution, regression networks generally use linear activation functions in their final layer. They are optimised with distance-based loss functions, such as Mean Squared Error (MSE) or L_1 loss, to minimise coordinate prediction error. Overall, the network looks very similar to the classification network, except for the output layers.

2.2.3 Segmentation Neural Network

Segmentation represents a significantly harder task. Instead of providing a single output for the entire image, the goal is to assign a prediction to every single pixel. Thus, the output must match the input image's spatial resolution (or a downscaled version that can then be upsampled). If there are C possible classes, the network produces an output tensor of size $H \times W \times C$, where H and W are the height and width of the image (See the third example of Figure 2.4).

This dense prediction task was revolutionised by Fully Convolutional Networks (FCNs) [27], which replace traditional fully connected layers with convolutional layers, allowing the network to output spatial maps. These networks typically employ an encoder-decoder architecture: the image is first downsampled to extract deep semantic features, then upsampled (e.g., via transposed convolutions) back to the original image size to map these features to individual pixels; this is illustrated in Figure 2.6.

It is important to distinguish between two main types of segmentation:

- **Semantic Segmentation:** Assigns a class label to every pixel (e.g., identifying all pixels that belong to a "dog"). However, if two dogs are standing side by side, they are merged into a single "dog" mask.
- **Instance Segmentation:** Not only assigns a class to each pixel but also differentiates between

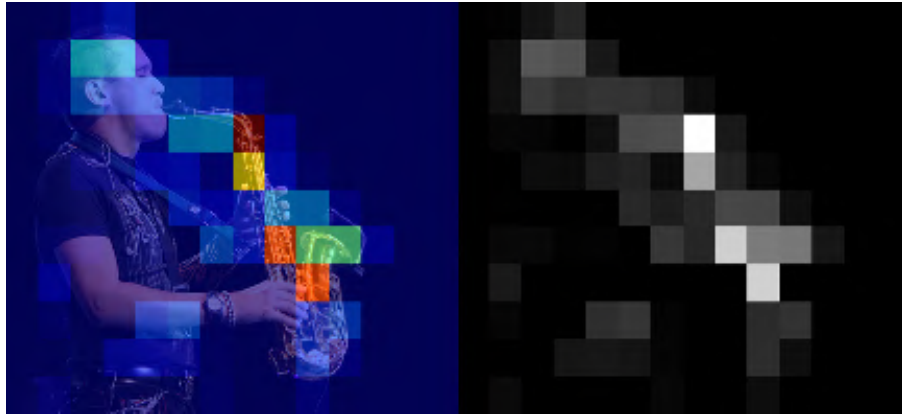


Figure 2.7: Occlusion: finding the relevant parts of an image by hiding parts of the image, Figure from [18].

separate objects of the same class (e.g., "dog 1" and "dog 2"). This requires the network to simultaneously perform pixel-level classification and object-level separation, making it the most challenging of the three paradigms.

This will be explained in further Section 2.4.

2.3 Explainable AI (XAI)

The goal of explainable AI is to clarify why an artificial neural network makes certain decisions. As deep neural networks become more prevalent in the real world, the ability to explain their outcomes has become increasingly important. For Example, the European Union now requires high-risk scenarios to be transparent, interpretable, and auditable. In the image classification use case, we aim to explain why the pixels led to this decision. There are several techniques to achieve this: Input Modification Methods, Deconvolutional Methods and Input Reconstruction Methods [18]. This XAI is relevant because most weakly supervised image learning methods also rely on it: now, with only a label, we want to localise the object, which is similar to the task where we want to know which pixels caused a certain activation.

2.3.1 Input Modification Methods

In input modification, explainability is completely independent of the network used (a black-box approach). The only thing used is the input and its given output. In the image case, this works as follows: We apply different augmentations to the same input image and examine the change in the output. Then we can see how a given region affects the prediction. For Example (see 2.7), if we are classifying instruments and we hide an important part. Our model drops from 80% for instrument classification to 50%. We know that part of the image is significant for classification. In one of the occlusion papers by Zeiler & Fergus (2013) [43], they overlaid a grey box on the image. Because of the significant impact of a CNN, Zhou et al. (2014) [47] later extended the Occlusion method by using a box filled with random pixels.

2.3.2 Deconvolutional Methods

In contrast to the previously explained method, this method is not a black-box method, and we will use the network to understand what led to the classification. Here, the idea is to determine the contribution of the pixels by starting with the activation of interest and going backwards through the network until the first input layer (the image). The result will be a mapping of how useful a certain pixel was for the classification task. For Example, given an image of a dog, the useful pixels could be those representing the ears or the nose.

Early techniques in this category include the Deconvnet approach by Zeiler & Fergus (2013) [43], which uses a multi-layered Deconvolutional Network to project activations from the feature space back to the input space. Another foundational method by Simonyan et al. (2013) [37] determines pixel importance by backpropagating the derivative of the class score to the input space. Ultimately, the primary differences among deconvolutional approaches lie in how they propagate these contribution values through ReLU and convolutional layers.

2.3.3 Input Reconstruction Methods

Input Reconstruction: An input that either maximally activates a unit of interest or leads to the same activations as a natural image reveals which features are important to the associated filters. Notable approaches in this category are [18]:

- **Gradient Descent in Input Space:** Simonyan et al. (2013) [37] used gradient descent in the input space that iteratively finds an image that maximally activates a chosen unit. They achieve this by backpropagating the error between the actual and desired activation values to the input layer, without altering the network.
- **Regularisation for Clearer Images:** Mahendran & Vedaldi (2014) [29] expanded upon the gradient descent method by introducing strong regularizers. They discovered that using an L_p norm (with higher values of p) alongside total variation improved the quality of the results, resulting in more natural-looking reconstructed images.
- **Feature Space Replacement:** Long et al. (2014) [28] evaluated whether CNNs learn interclass correspondence by replacing patches of the input image with patches from other images (top- k nearest neighbours) that lead to the same activations at the layer of interest.
- **Generative Networks:** Dosovitskiy & Brox (2015) [13] used generative networks (essentially CNNs turned upside down) to invert the image representation. While these networks require training to invert a specific representation, they offer the benefit of computing reconstructions several orders of magnitude faster than gradient-based methods.

2.4 Localisation and Segmentation

While the difference between localisation and segmentation is subtle, we must clarify what it is in the current weakly supervised context. In localisation, the goal in most papers is to draw a bounding box around the object; the ideal bounding box must enclose the entire object while being as small as possible. Of course, for Example, with a very spread-out object, e.g., a bike, the bounding box is large, but many pixels inside it will not be part of the bike. With segmentation, the goal here is different: we want an accurate pixel-wise mask of the object.

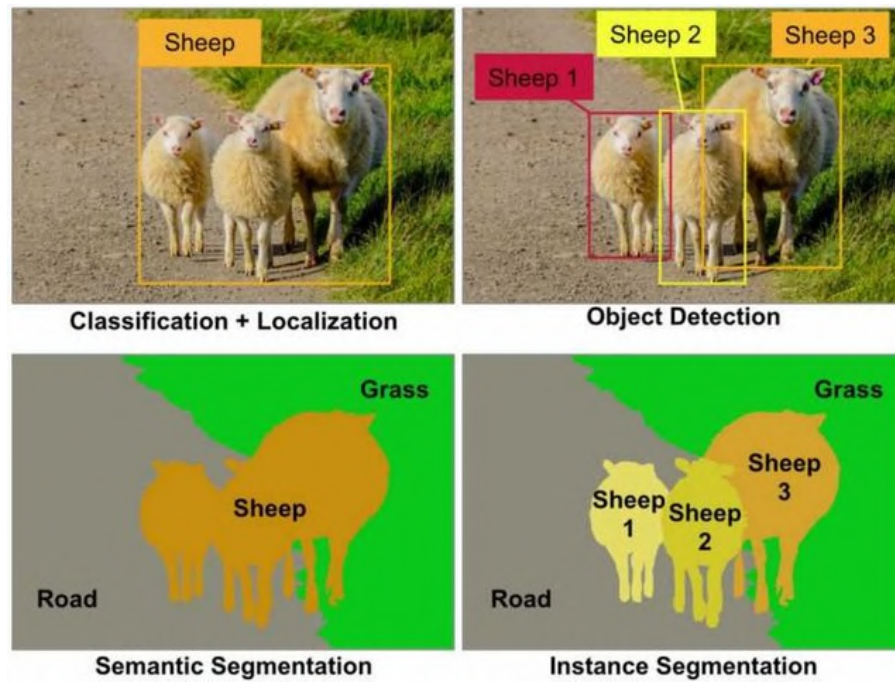


Figure 2.8: Distinction between a localisation, detection, semantic segmentation, and instance segmentation model. Figure from [33]

2.5 Landscape of Image Supervision Techniques

2.5.1 Output types

There are several different model output goals in this research field: Object localisation models [12] [36] [46] [45] [42] [41], Object detection models [15] [17], Semantic segmentation models [9] [21], and Instance segmentation models [20] [25] [12] [5]. All have a similar goal: finding the location of an object. Object localisation is the simplest; here, the goal is to localise a single object of a single class using a bounding box or binary mask. Object detection models make it a bit more complicated; instead, here we want to draw multiple bounding boxes on different classes and instances. In semantic segmentation models, the output is a pixel-wise class map. And finally, in instance segmentation, the goal is to produce a pixel-wise class-instance map (so multiple items of the same class must have distinct annotations).

2.5.2 Amount of Supervision

The amount of supervision can be split up into three categories: Supervised, weakly supervised, and unsupervised. In a supervised learning setting, the model is trained on examples that match what the network must generate during inference. This is the most commonly used technique, but it has the disadvantage that creating image annotations requires substantial manual labour. To resolve this, weakly supervised learning is used to minimise annotation time. For Example, instead of using pixel-mask annotations, only image labels are used; this, in combination with some novel ideas (Often based on XAI methods in image use cases, see Section 2.3), can still train a segmentation model. Finally, there is unsupervised learning [16], which is trained with an unannotated dataset. Methods like K-means clustering, which group data based on how close the points are, and the Apriori Algorithm, which finds patterns by exploring frequent item combinations step by step. Of course, the supervision you need

depends on the specific use case, dataset, goal, etc. For localisation and segmentation tasks, these methods are supervised and weakly supervised.

2.5.3 WSOL, WSSS, WSIS

WSOL (Weakly Supervised Object Localisation) is a technique in which, using image-level labels for images, we localise objects by finding one instance and drawing a bounding box around it. With WSSS (Weakly Supervised Semantic Segmentation), the goal is to create a mask for multiple object classes; however, if there are multiple instances of the same class, they will be represented by a single pixel-wise mask. Finally, WSIS (Weakly Supervised Instance Segmentation) will work with limited labels (This can be more than just an image-level label; for example, also a location point, depending on the amount of supervision) to generate instance-wise masks, ensuring that every instance of the same class has its own pixel-wise mask. Sometimes papers use other abbreviations, but the above are the most commonly used, and there is a clear pattern:

- **WS (first two letters):** Stands for Weakly Supervised, meaning we don't have examples of the output but only limited alternative labels.
- **O (third letter):** Object, which most of the time indicates there is only one instance in every sample.
- **S (third letter):** Semantic, here we try to split into the correct classes.
- **I (third letter):** Instance, representing that we will split instances even from the same class.
- **L (third letter):** Localisation, we try to localise the item in the image by using a bounding box.
- **Sv (fourth letter):** Segmentation, we try to create a perfect pixel-wise mask of the item.

2.6 Dataset Annotation

As explained in the previous section, depending on the amount of supervision, certain dataset annotations are needed. For image localisation and segmentation tasks, often Caltech-UCSD Birds-200-2011 (CUB-200-2011)[39], MS COCO [26] or Pascal VOC 2012 [14] are used. This is because they contain many images with multiple annotations, labels, and instance pixel masks. There are different forms of image annotations, some of which take more time than others: Instance pixel masks, Image-Level labels, Point-Level labels, and Bounding-box labels.

2.6.1 Instance Pixel Mask Annotations

For full supervision, we have a pixel mask and a label for every instance in the image. To annotate this dataset it requires 221.76 sec/img [6] this result comes from following calculation: It takes 22 work-hours to produce 1000 segmentations [26]. Thus $(3600s * 22) / 1000 = 79.2s$ per segmentation. Knowing there are on average 2.8 object instances, and bringing our total to 221.76s/img. For datasets with thousands of images, for example, the COCO dataset with more than 200.000 labeled images, this takes a large amount of manual labour.

2.6.2 Image-Level Labels Annotations

With image-level supervision, the only additional information provided is the labels indicating what is present. The paper by Dim P. Papadopoulos et al. [32] attempts to draw bounding boxes around objects using eye-tracking information. During their experiment, they also collected this information about how long it takes to find an object in an image and to press a button. This process takes approximately one second. From this, the assumption is that it takes one second per class in the dataset [6]. Thus, for VOC 2012 (20 classes), this results in 20.0 sec/img, or for COCO 2017 (80 classes), 80.0 sec/image.

2.6.3 Point-Level Annotations

Here we have a point and a label for every instance in the image, providing a hybrid between Full Supervision and Image-Level Supervision. In addition to the label, we also have a point on the instance that shows the location. This offers more information while still being relatively fast to annotate the dataset. The Semantic Segmentation with Point Supervision paper [32] reports an average annotation time of 22.1 sec/img for the VOC 2012 dataset, which is around 10 times faster than full supervision and only 2 sec slower than image-level supervision.

2.6.3.1 Bounding-Box Annotations

Drawing bounding boxes around every instance of an image will take more time than point-level annotations, but it will take significantly less time than complete pixel map annotations. In the paper "Crowdsourcing Annotations for Visual Object Detection" [38], the authors found that it takes a median of around 26 seconds to draw a bounding box. This for VOC would result in an average of ± 62 seconds per image, because VOC contains 11,530 images with 27,450 annotated ROI objects: $(27450 * 26) / 11530$. which takes almost 3 times as long as point annotations, but still 3.6 times faster than pixel map annotations.

3 Methodology

3.1 Overview

In our work, we will focus on Background Activation Suppression (BAS), Beyond Semantic to Instance Segmentation (BESTIE), and Complete Instances Mining (CIM) because they are relatively recent and all have been published. While IRNET [5] would be interesting, as it is one of the first papers that does WSIS, but is outperformed by BESTIE and CIM, and is significantly older (from 2019; BESTIE is from 2022; CIM is from 2024), for this reason, we opted not to include this one. There are also other WSIS papers like BoxTeacher [11], DiscoBox [22], Pointly-Supervised Instance Segmentation [10], BBAM [23]; we didn't use these because they use extra or different data than only the image-level labels, for Example, bounding boxes and points, while this is still weakly supervised, as the masks are not given in this work. We will only focus on weakly supervised methods that use image-level labels. All used code can be found in my thesis GitHub repository <https://github.com/SteinVandenbroeke/thesis> or in the referenced repositories in this repository.

3.1.1 Comparing and Testing the Methods

After finding WSIS methods, we decided to run our experiments on Background Activation Suppression (BAS), Beyond Semantic to Instance Segmentation (BESTIE) and Complete Instances Mining (CIM). BAS was originally created as a WSOL method and later, in a follow-up paper, also used as a WSSS method. This allows us to use these methods as a perfect baseline method. In the experiments, we will compare other WSIS methods against a basic WSSS method; this allows us to assess whether the methods improve results and whether the currently used metric accurately represents the WSIS methods. Figure 3.1 shows an abstract pipeline illustrating how we tested multiple methods in the same way. As expected, we split our datasets into train/test. Every model is trained on the train dataset and evaluated on the test set. These arrows are not shown in the Figure to keep it comprehensible. To make it easy to store all results in a manageable, easy-to-transfer format, we saved them in the COCO JSON format. This also allowed us to use a single evaluation script with minimal changes across the COCO and VOC datasets, since they now share the same format. As you can see, the output of the BAS WSSS feeds into the input of BAS WSIS; this will be explained further in Section 3.4.1. Because the VOC and COCO datasets were not originally used on all methods (or the code was not shared), we had to implement custom dataloaders for some of the methods; even with the more recent methods, there were still a lot of challenges getting these running because multiple methods used deprecated packages or deprecated Python versions. Because of this, we created a fork of each method with the necessary changes to run the experiments (the required changes and challenges are discussed in detail per method in Chapter 4).

3.1.2 Testing whether the WSIS approach has an impact on a WSOL task

To test the impact of a WSIS method in a WSOL setting (single instances on each image and bounding boxes instead of masks). We used the CUB dataset, which contains only single-instance images (one bird per image). We used the BESTIE method and BAS method to compare the outputs. If the WSIS

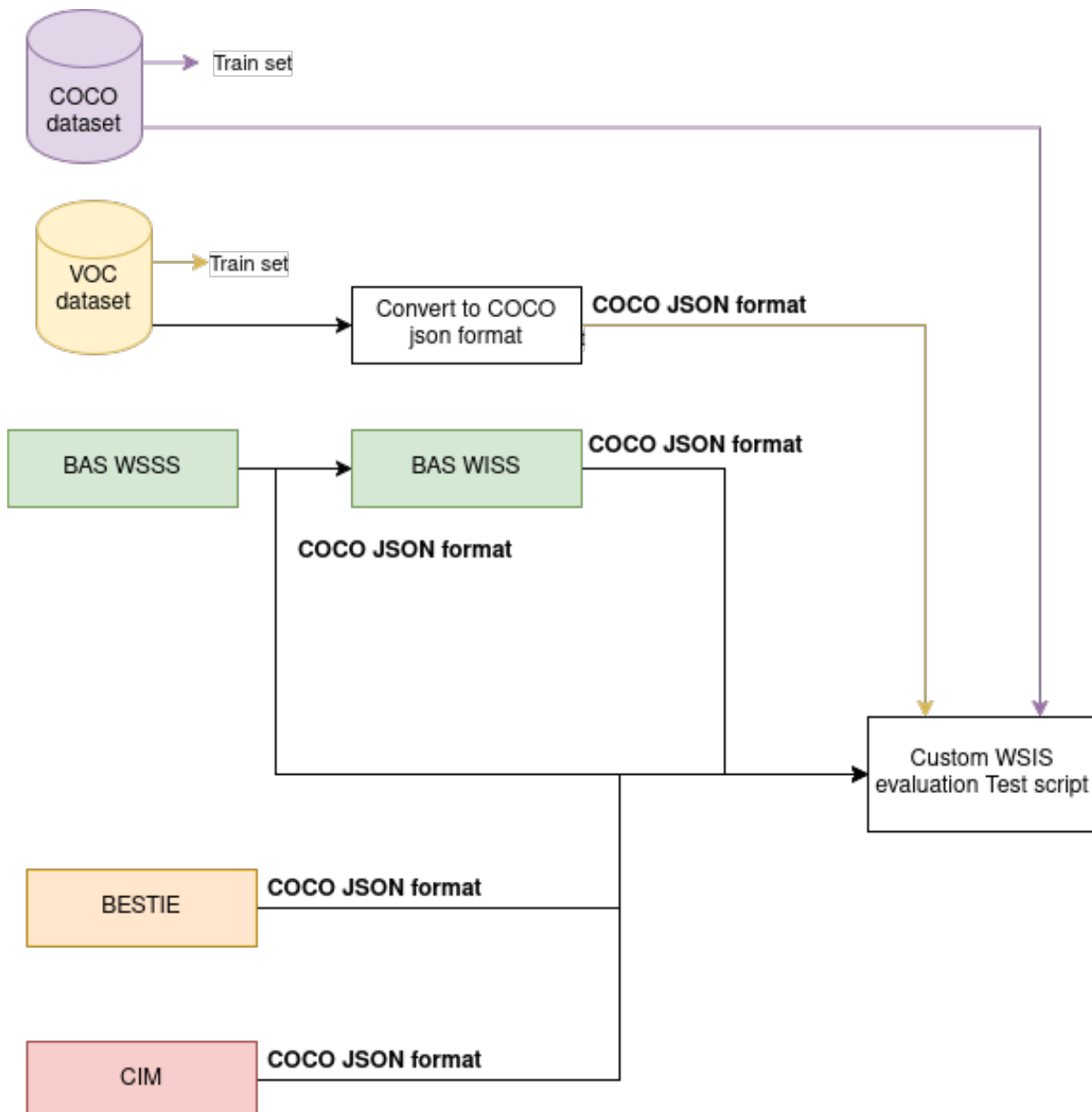


Figure 3.1: Figure showing a simplified flow of how the different methods were compared to each other. Using a general script and by converting all model evaluation outputs to a COCO JSON format, we could easily compare them.

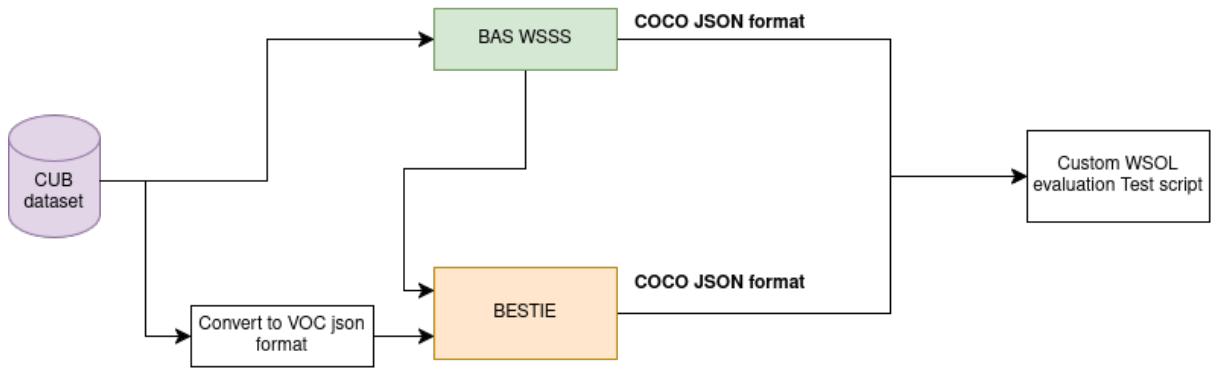


Figure 3.2: Figure showing a simplified flow of how the WSIS method was tested on a WSOL problem.

method has no impact, the result of the custom WSOL evaluation script would be the same as with BESTIE, which will use the custom WSSS masks from the BAS approach as input. This flow can be seen in Figure 3.2

3.2 WSOL methods

In Section 3.4, we discussed that Weakly Supervised Object Localisation (WSOL) tries to localise an object using only image labels. This means the goal is to locate a single object using, for example, a bounding box. Only one method will be discussed, as the research focus shifted towards WSIS.

3.2.1 Background Activation Suppression (BAS)

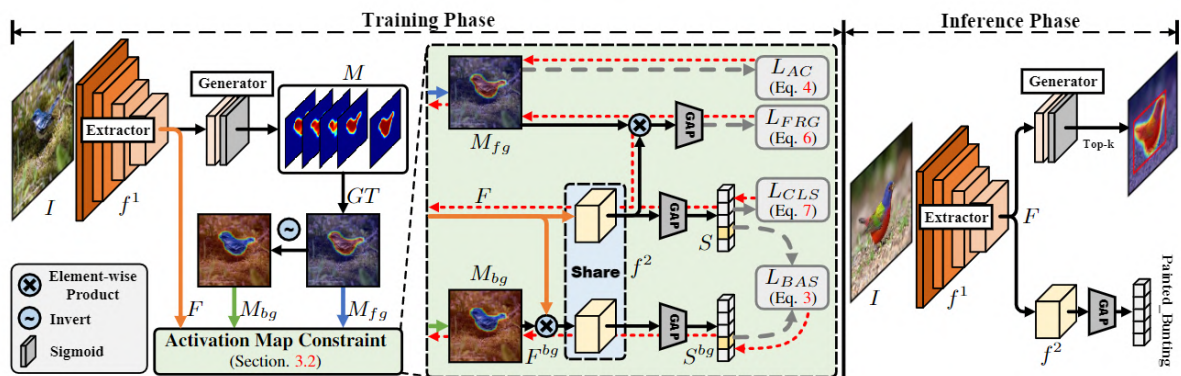


Figure 3.3: Layout of the Background Activation Suppression method [41]. Showing the three main modules and the four loss functions.

Other methods that use Class Activation Maps (CAM) commonly struggle here because they tend to focus on the most discriminative parts of an object (such as a bird's beak, or only the face of the cat instead of the full body; see Figure 3.4) rather than the entire object.

In the paper "Background Activation Suppression for Weakly Supervised Object Localisation" [41], Wu et al. address this issue by moving away from conventional CAMs and using a Foreground Prediction Map (FPM) [30] instead. They noticed a major problem with standard Cross-Entropy (CE) loss: CE loss drops to zero as soon as the foreground mask covers even a small, easy-to-recognise part of the



Figure 3.4: Example of the CAM created for a cat image. Image generated by a model created when we were still exploring the original research questions.

object. On the other hand, the network's activation value continues to increase until the mask reaches the object's actual boundaries. Since directly maximising activation values is difficult for deep neural networks, the BAS method takes the opposite approach: it helps the network learn the foreground map by suppressing background activation values. The BAS architecture does this using three main modules:

- **Extractor (f^1):** This is just the first part of a Convolutional Neural Network (CNN) acting as a feature extractor. It takes the input image and produces a base feature map F that contains high-level spatial information about both the foreground object and the background.
- **Generator:** Next, the feature map F is fed into a generator that produces a class-specific foreground prediction map (M_{fg}). This map shows where the network thinks the object is. The network then mathematically inverts M_{fg} to get the opposite background prediction map (M_{bg}).
- **Activation Map Constraint (AMC):** This is the most important module of the BAS method. Here, the feature map F , the foreground map M_{fg} , and the background map M_{bg} are used to mask the features. These separated features are then passed through the second half of the shared network (f^2) to compute the final activation values.

To make sure the mask covers the entire object, the AMC module uses four different loss functions during training:

- **Background Activation Suppression Loss ($\mathcal{L}_{BAS}$):** This is the main driver of the network. It takes the background activation value (S^{bg}) and minimises its ratio against the overall image activation value (S). If the foreground mask only covers a bird's beak, the rest of the bird's body leaks into the background map. This causes a high S^{bg} and a heavy penalty. To fix this and minimise the loss, the network is forced to expand the foreground mask M_{fg} until it covers the whole object, leaving no object features in the background.
- **Area Constraint Loss ($\mathcal{L}_{AC}$):** If the network relied solely on $\mathcal{L}_{BAS}$, it could easily cheat by expanding the foreground mask to cover the entire image, leaving the background completely empty. To prevent this, the area constraint loss penalises the network when the foreground prediction area becomes too large, forcing the background area to remain as large as possible.
- **Foreground Region Guidance Loss ($\mathcal{L}_{FRG}$):** This loss uses standard high-level semantic classification (Cross-Entropy) to guide the foreground prediction map to the roughly correct region of the object during the early stages of training (for Example, by finding the easiest parts first).
- **Classification Loss ($\mathcal{L}_{CLS}$):** This is a standard classification loss applied to the unmasked image features for learning how to classify the image correctly.

By equilibrating these opposing forces— $\mathcal{L}_{BAS}$ pushing the mask outward to cover the whole object, and

$\mathcal{L}_{AC}$ pulling it back to avoid covering the background- the generator naturally generates the foreground map around the edges of the object. And it does this without needing any pixel-level or bounding-box annotations. Of course, while this was originally designed for WSOL, its ability to generate high-quality full-object prediction maps makes it an ideal base for Weakly Supervised Semantic Segmentation (WSSS) and Weakly Supervised Instance Segmentation (WSIS), which will be discussed in the next sections as well.

3.3 WSSS methods

Here, the goal is to create masks for multiple classes, but each class gets only one mask assigned, as already explained in more detail in Section .

3.3.1 Background Activation Suppression extended (BAS WSSS)

The background activation suppression method was already explained in Section 3.2.1. In the follow-up extension paper by Wu et al. [44], the authors adapt the BAS method to perform WSSS, creating pixel-wise masks for multiple classes in a single image.

To achieve this, the method was modified to handle multi-class datasets. The generator now outputs a set of foreground prediction maps $M \in \mathbb{R}^{H \times W \times C}$ (with a 0-1 distribution), where C is the total number of available categories instead of the original single-channel $M \in \mathbb{R}^{H \times W \times 1}$ output from the WSOL generator (see Section 3.2.1). This provides a foreground prediction for every individual possible class.

During the training phase, a class-specific foreground prediction $M_f \in \mathbb{R}^{H \times W \times 1}$ is randomly selected from the ground-truth classes present in the image. This map is then inverted to create the background mask M_b . While these masks are fed into the Activation Map Constraint (AMC) module similarly to the original BAS pipeline, there is one crucial modification: because the network must now handle multiple labels simultaneously, the standard classification loss ($\mathcal{L}_{cls}$) is replaced by a multi-label classification loss ($\mathcal{L}_{mcls}$). The authors use a modified softmax cross-entropy rather than a sigmoid-based loss to avoid mathematical conflicts with the non-negativity requirement of the background activation value (S^b).

Finally, it is important to note that the extended BAS network itself serves as a high-quality "seed generation" step. To accomplish the final WSSS task, these generated pseudo-label masks are refined (using techniques like IRN) and then used to train a fully supervised semantic segmentation network, such as DeepLabV2.

3.4 WSIS methods

In this section, we go over the WSIS methods used (see Section).

3.4.1 Background Activation Suppression extended modified (BAS WSIS)

There is no paper on the use of BAS in an instance segmentation setting. We decided to use this method as a baseline by splitting a class mask into multiple masks when the masks are not connected. This is a very primitive way of handling instance segmentation because, when an object obstructs an instance,

the instance mask is split into two masks for the same instance, or when two instances are closely overlapping, it results in a single mask.

3.4.2 Beyond Semantic to Instance Segmentation (BESTIE)

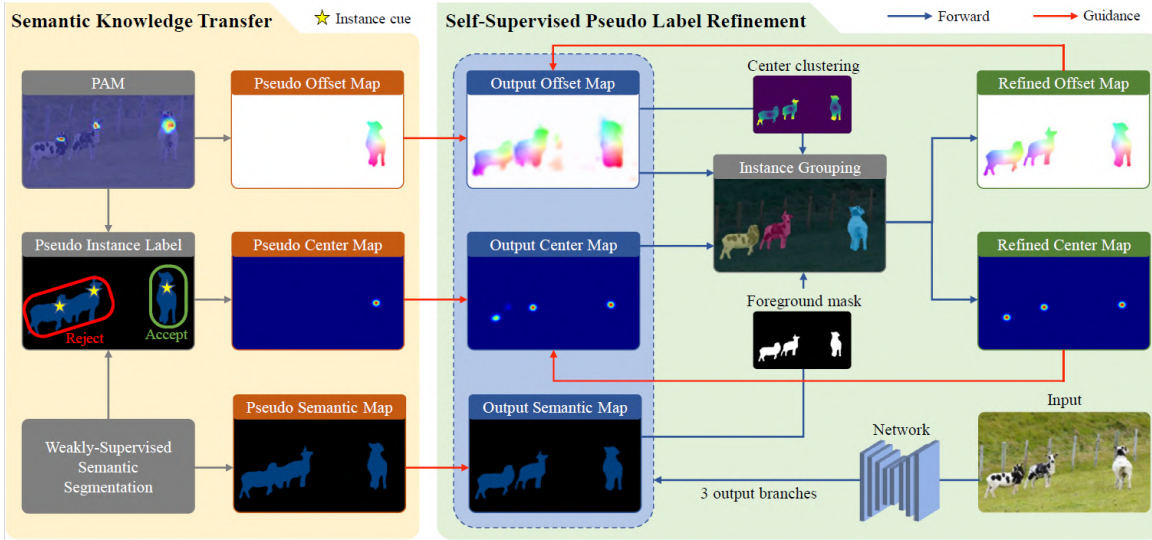


Figure 3.5: Overview of the working of the BESTIE approach. Figure from [20].

In this study, we propose to utilise the BESTIE (BEyond Semantic segmentation To Instance Segmentation) framework. This approach is specifically designed for Weakly-Supervised Instance Segmentation (WSIS), relying exclusively on image-level labels and strictly following a fully image-level supervised setting by deprecating reliance on off-the-shelf proposal techniques (which regularly require pre-training with instance- or object-level labels). Furthermore, BESTIE effectively tackles the semantic drift problem, in which missing instances in pseudo-labels are incorrectly categorised as background, confusing network training. The BESTIE framework consists of two core innovative components: *Semantic Knowledge Transfer* and *Self-Supervised Pseudo Label Refinement*.

3.4.2.1 Semantic Knowledge Transfer

The first component, Semantic Knowledge Transfer, solves the instance segmentation by transferring knowledge from Weakly-Supervised Semantic Segmentation (WSSS) to WSIS to generate initial pseudo-instance labels. This is done using the following key steps:

- **Peak Attention Module (PAM):** Traditional Class Activation Maps (CAMs) create noisy activation regions, resulting in multiple inaccurate instance cues for a single object. To prevent this, BESTIE introduces PAM to extract appropriate instance cues from image-level labels. PAM strengthens the network's attention to the sparse, representative peak regions of an object while mathematically deactivating noisy regions. It consists of a selector, a controller, and a peak stimulator to set the boundary of peak regions dynamically.
- **Pseudo Instance Mask Generation:** Using the WSSS output and the instance cues extracted by PAM, the framework identifies non-overlapping instances. Applying the connected component labelling (CCL) algorithm to the WSSS output generates instance mask candidates. If a mask candidate contains exactly one instance cue, it is accepted as a valid pseudo instance mask.

- **Instance Representation:** The accepted pseudo instance masks are then converted into a pseudo centre map (identifying the centroid of the object) and a pseudo offset map (containing 2D vectors directing all pixels in the mask to their corresponding centre point).

3.4.2.2 Self-Supervised Pseudo Label Refinement

The initial pseudo instance labels generated are often incomplete, containing many false negatives (missing instances). To continuously improve label quality during training, BESTIE uses a self-supervised refinement strategy.

- **Instance-Aware Guidance:** To prevent the semantic drift problem, BESTIE dynamically restricts the guidance region for the offset and centre maps exclusively to the explicitly labelled instances, treating the background as an ignored region. By not penalising the network for missing-feature instances in the background, the network can learn stably and progressively capture instance-level information for those unmapped objects as training progresses.
- **Online Refinement and Centre Clustering:** As the network's generalisation ability improves, it groups the pixels into instances based on its own predictions. It generates refined offset and centre maps from these grouped masks. To further enhance this process, a centre clustering algorithm groups the 2D offset vectors to accurately locate centre points, complementing any points missed by the output centre map.
- **Feedback Loop:** The newly discovered instances are converted into high-quality refined labels and are utilised as soft labels in the objective function alongside the pseudo labels. This self-supervised conversion of false negatives into true positives occurs online at each mini-batch, improving the training guidance.

By using Semantic Knowledge Transfer to localise reliable instance cues, along with Instance-Aware Guidance and Online Refinement, the BESTIE framework creates accurate instance segmentations without requiring bounding-box or pixel-level annotations.

3.4.3 Complete Instances Mining (CIM)

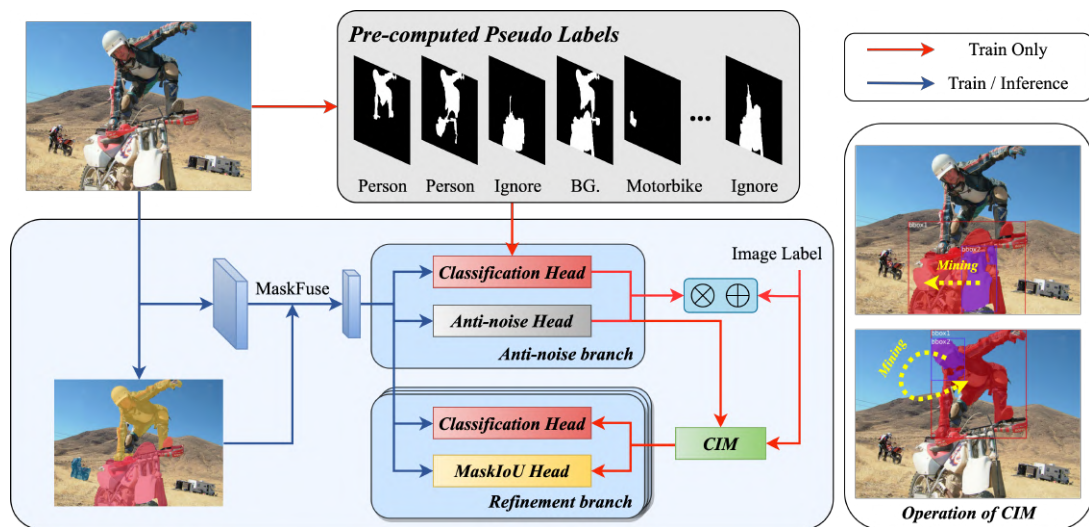


Figure 3.6: Working of CIM Figure from [25]

In addition to BAS and BESTIE, we will use the CIM (Complete Instance Mining) framework [25]. CIM is specifically designed for Weakly-Supervised Instance Segmentation (WSIS). Furthermore, CIM effectively tackles the redundant segmentation problem, in which a single instance is incorrectly represented by multiple proposals, leading the network to predict multiple discriminative parts rather than a single complete object. The CIM framework consists of three core innovative components: MaskIoU Heads, a Complete Instances Mining strategy, and an Anti-noise strategy.

3.4.3.1 Refinement Branch and Complete Instances Mining

The first major component solves instance segmentation by identifying complete instances from a redundant set of proposals, thereby generating refined pseudo-labels. This is done using the following key steps:

- **MaskIoU Heads:** To learn the quality of predicted masks, the CIM uses a MaskIoU head. Given proposal features extracted via a lightweight MaskFuse operation, the refinement branches produce both classification scores and integrity scores (MaskIoU) for each proposal.
- **Seed Selection:** CIM models the redundant segmentation problem by noting that partial segmentations typically have high classification scores but low integrity scores, while complete segmentations are the opposite. The framework uses these scores to select initial seeds, assuring the network remains aware of multiple instances and maintains a high recall.
- **Pseudo Ground Truth Mining:** Because partial segmentations are often spatially contained within complete ones, the framework takes each seed and finds the containing proposal with the highest integrity score. This chosen proposal is accepted as a valid pseudo-ground truth, allowing the method to aggregate seeds and effectively mine complete instances.

3.4.3.2 Anti-Noise Strategy

The precomputed pseudo labels (often used to warm up the model) and the newly refined pseudo labels generated during training often contain noise, which can confuse the network and lead to overfitting. To continuously improve robustness, CIM employs an anti-noise strategy.

- **Anti-Noise Branch:** The framework implements a branch that is supervised by the precomputed pseudo labels, but its dedicated anti-noise head is not directly guided. This lack of direct guidance allows the redundant parameters in the head to effectively filter out the initial noise from the precomputed labels.
- **Anti-Noise Sampling:** The refined pseudo labels can also be noisy because of relaxed constraints. Noting that noisy labels tend to have lower loss weights, the framework uses these loss weights as sampling probabilities. Performing sampling with replacement will filter out noise from the refined pseudo-labels.

By using MaskIoU heads and the Complete Instances Mining strategy to resolve redundant segmentations, along with an Anti-noise strategy, the CIM framework produces accurate instance segmentations that are unaffected by noisy pseudo-labels.

4 Experiments and results

First, we will go over the metrics used in the experiments. Then we will go over the datasets we used and run some tests on them. Finally, we will go over the results of these (custom) metrics applied to the methods discussed in Section ref Chapter: Methodology.

4.1 Metrics

4.1.1 Intersection over Union



Figure 4.1: Three examples of the IoU values for 3 cases. Reference [35]

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Intersection over Union, or in short IoU, is often used as a localisation metric; it measures how well a predicted bounding box or mask aligns with the ground truth. It is calculated by first taking the intersection of the ground truth and prediction and dividing this by the Union of the ground truth and prediction. If they match, the IoU score will be one because the intersection will then completely match the Union. Any other combination will result in a value less than 1 because the intersection area can never exceed the union area. In Figure 4.1, the working is visualised.

4.1.2 Mean Average Precision

mAP (mean Average Precision) is an overall metric for measuring performance, while IoU is a metric for measuring performance on a single instance. mAP allows us to obtain a single score for the entire dataset. The mAP score is calculated as follows: first, an IoU threshold is set, such as 0.25, 0.5, 0.75, etc. Then, for each image, the IoU is calculated; if this value exceeds the set threshold, the prediction is classified as a True Positive (TP). Conversely, if the IoU falls below the threshold, or if a predicted bounding box or mask has no corresponding ground truth, it is deemed a False Positive (FP). Any actual ground truth instances that the model fails to detect are recorded as False Negatives (FN). Using these classifications across the dataset, we compute two fundamental metrics for each class: precision, the proportion of positive predictions that are actually correct, calculated as $\frac{TP}{TP+FP}$. Recall: The proportion of actual ground truth instances that were correctly identified, calculated as $\frac{TP}{TP+FN}$. By evaluating the

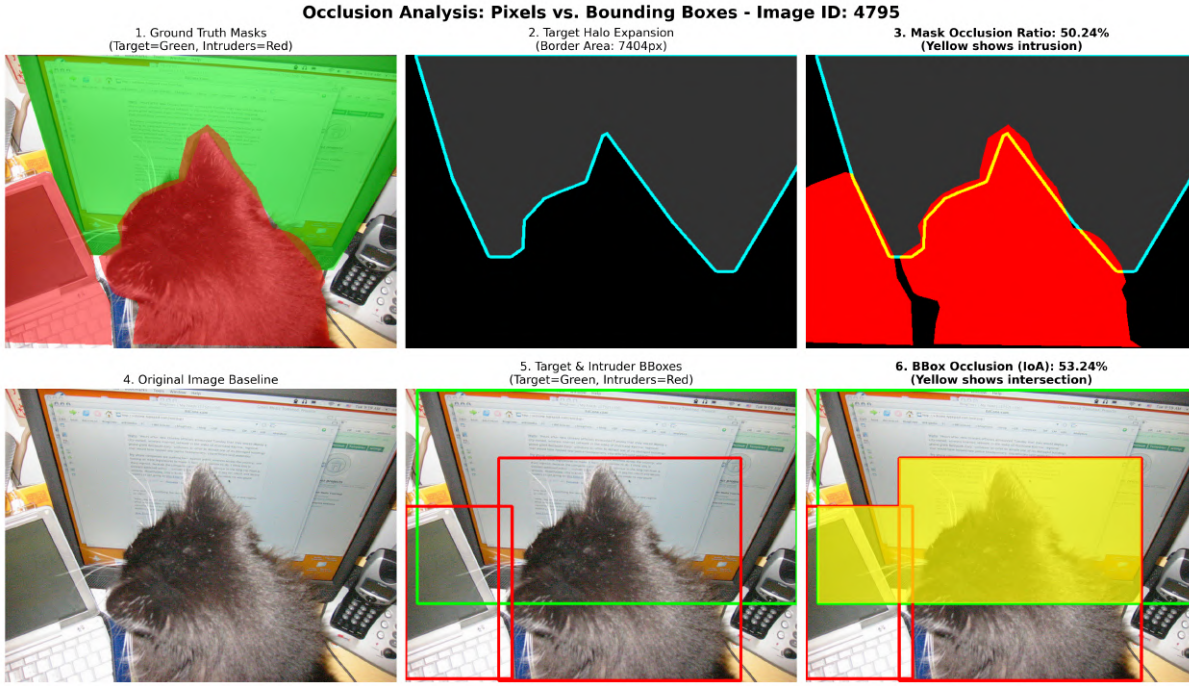


Figure 4.2: Occlusion metric by using bordering pixels (upper images) and using bounding boxes (lower images).

model at different confidence levels, we obtain a series of precision and recall values. These values are plotted against each other to form a Precision-Recall (PR) curve. The Average Precision (AP) for a single class is defined as the area under this PR curve, which is mathematically expressed as:

$$AP = \int_0^1 p(r)dr$$

where $p(r)$ represents the precision at a given recall level r . Finally, to obtain the Mean Average Precision (mAP) for the entire dataset, we calculate the arithmetic mean of the AP scores across all N object classes:

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$$

This final calculation gives a comprehensive, single-value summary of the model's localisation and classification performance among all categories.

4.1.3 Occlusion Metric

None of the datasets we will use was created specifically to test how well the methods could split instances. Thus, we must analyse the datasets thoroughly to ensure they are sufficiently challenging to test a WSIS method. The really interesting cases are the ones where we have to split overlapping instances of the same class because when items are of a different class we can use a semantic segmentation method. If we want to take the overlap into account, we first need to define overlap. This might seem obvious: take the intersection of two instances' masks, define this as their total mask size, and we have a result. There is one problem with this: we don't have the full instance masks. The masks were created from 2D images, so they are only drawn for the visible part; thus, 2 overlapping instances will still have no overlapping mask. To solve this problem, we have two possible approaches: count the pixels that touch the mask's border, or use bounding boxes to compute their overlap.

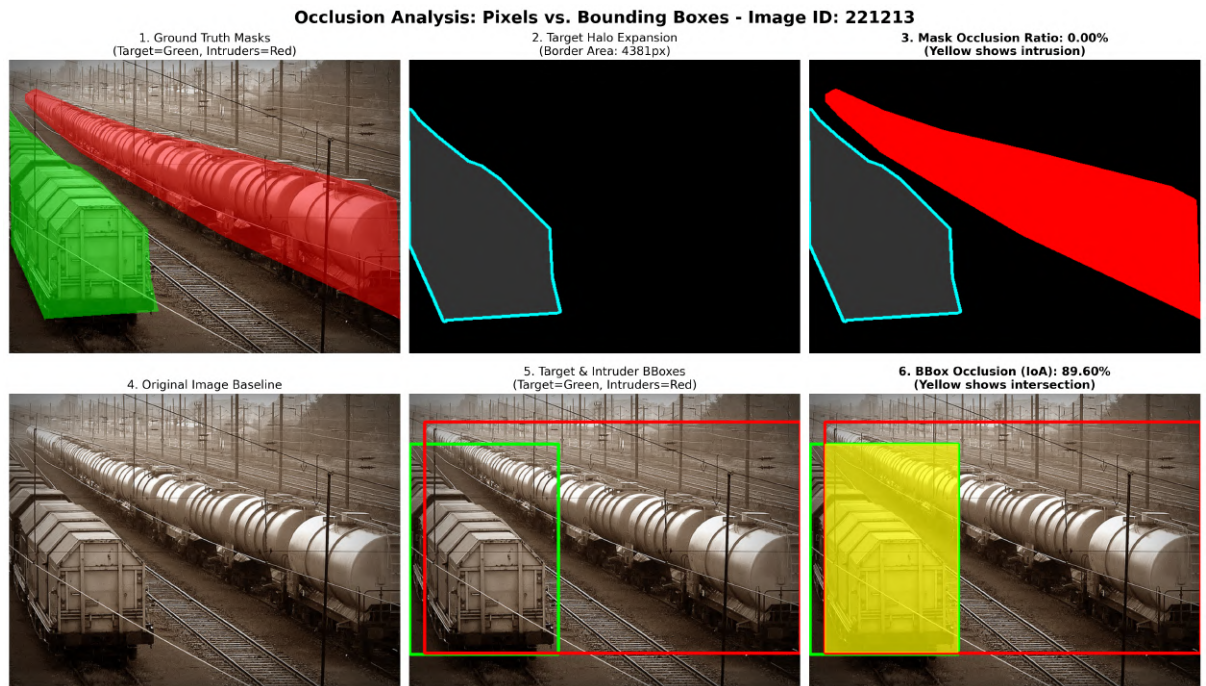


Figure 4.3: Occlusion metric by using bordering pixels (upper images) and using bounding boxes (lower images).

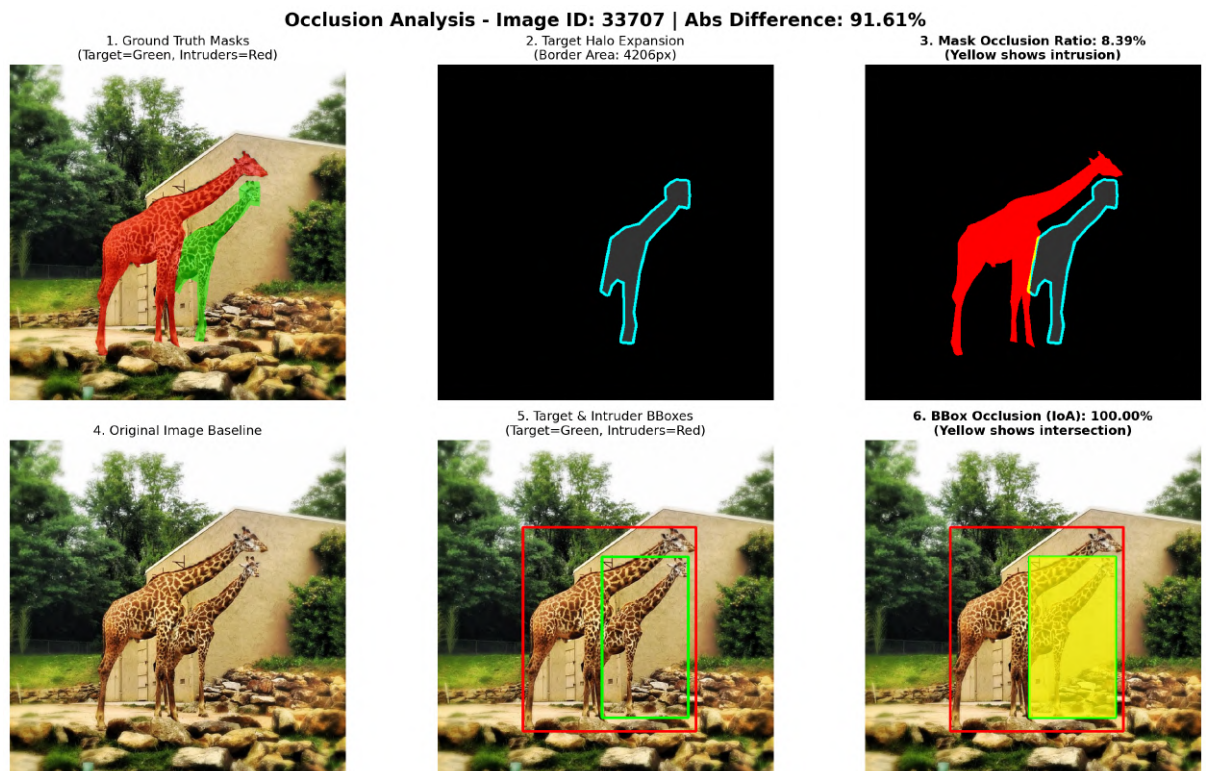


Figure 4.4: Occlusion metric by using bordering pixels (upper images) and using bounding boxes (lower images).

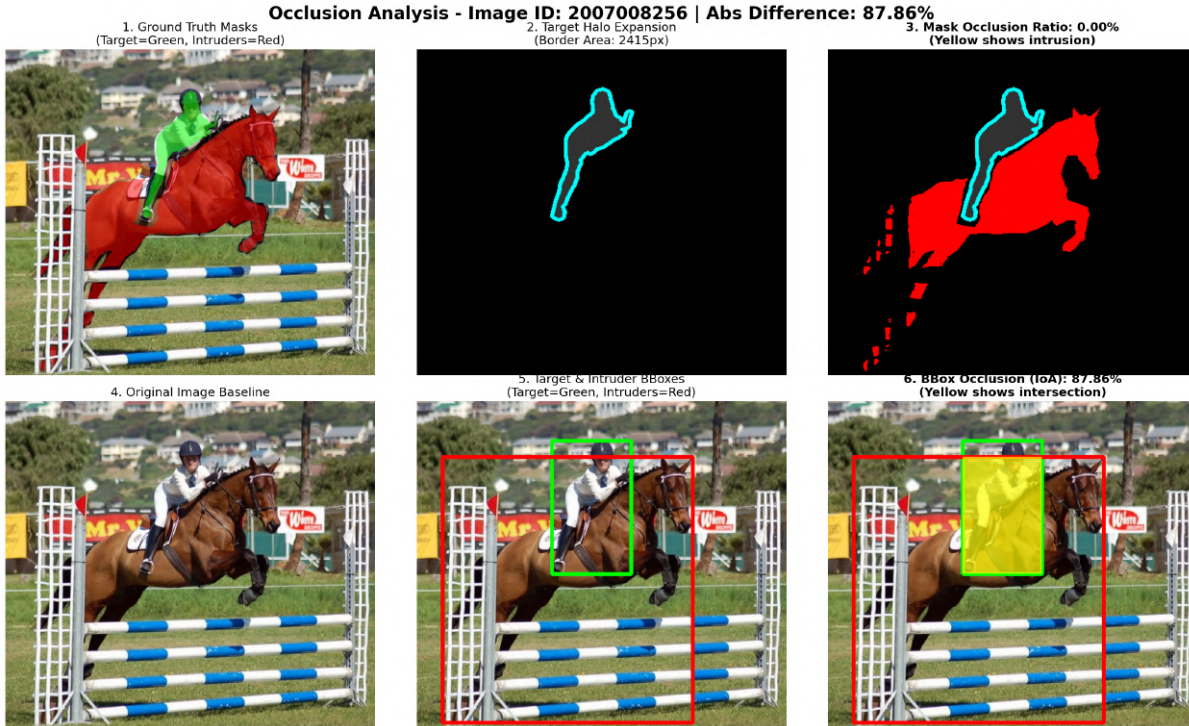


Figure 4.5: Bad annotated instances break occlusion on border pixels

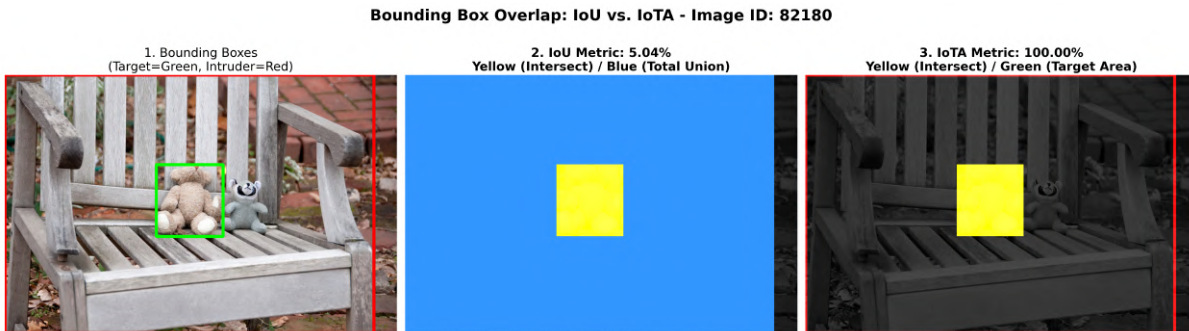


Figure 4.6: Example of why IoU does not work and why we should use IoTA (intersection of target area) for the occlusion metric between instances

4.1.3.1 Calculating Occlusion using Bounding Boxes

Here we use the bounding boxes of both instances to find an overlap value. Just using IoU will not work in this particular case because, for example, a small item placed on top of a big item has full occlusion and thus 100% IoU, but this would not be the case with IoU, which divides the overlap by the Union. To solve this, we divide by the target area; this is shown in Figure 4.6, where we want to calculate the occlusion of the bear by the chair. In the middle image, using IoU we get 5.04%, and with IoTA (Intersection over Target Area) we get 100%, which is the expected value because the chair completely encloses the bear. With AoTA, we still encounter a problem: with a large, compact object, for Example, a bike, the mask is significantly smaller than the bounding box, and thus two objects that do not overlap could be incorrectly given a high IoTA value indicating they overlap. This can be seen in the lower images of Figure 4.3, where the trains are long and diagonal; because of this, the bounding boxes are significantly larger than the trains, and thus the bounding box occlusion is high even when the trains are not actually overlapping or touching. Or in Figure 4.4 with giraffes.

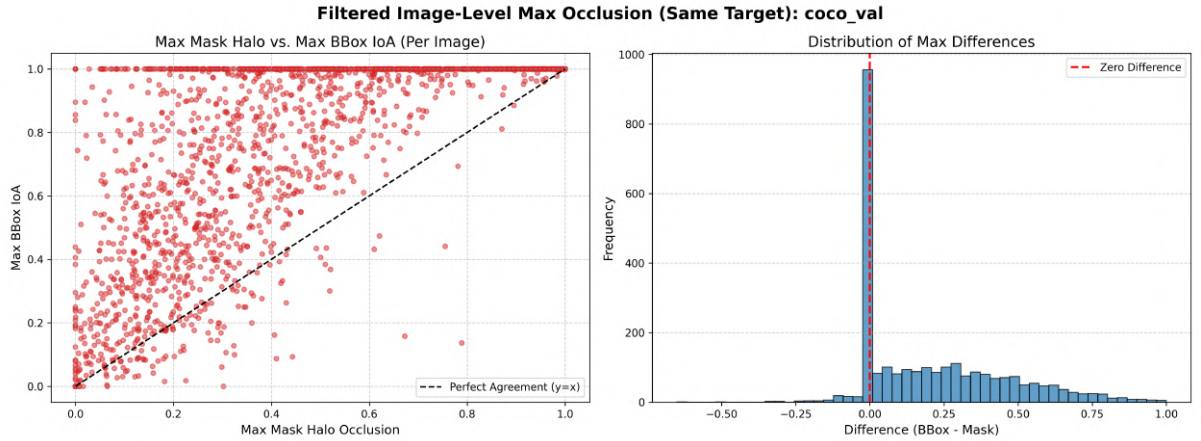


Figure 4.7: Figure showing the impact of using the bounding box metric compared to the pixel boundary metric on the COCO dataset to measure the occlusion for images.

4.1.3.2 Calculating Occlusion using Bordering Pixels

To solve the bounding box problem, we tried the following approach instead of using bounding boxes: we use the bordering pixel count. To do this, we have multiple steps:

1. We draw an outward border around the given gt segmentation; this will create overlap with the other instances touching the instance.
2. Go over every segmented instance in the image and match these with the newly generated pixel border
3. The number of matching pixels is divided by the total number of pixels in the border to get the occlusion score

This completely solves the problem of the too-large bounding boxes in the VOC dataset and makes the occlusion metric significantly more precise. Of course, now we have the problem that if an instance is contained within another instance, the border of the larger instance doesn't overlap with the smaller one, but this will not be a problem, as we will take the worst occlusion score of the two. This can be seen in Figures 4.3 and 4.4 on the upper image rows. Here, the occlusion values are significantly lower than those for the original bounding boxes. This seems like the perfect approach, but there is a major problem: some GT segmentations in the VOC and COCO datasets aren't perfect, which can lead to situations where the pixel boundary is not wide enough to overlap with bordering instance segmentations, so the overlap is not measured. This can be seen in Figure 4.5. Because this is a flaw in the dataset annotations, it's very hard to know whether they are two non-touching instances or badly annotated images. From checking the created samples in the VOC dataset, this occurs more often than in the COCO dataset, but it is still a limited problem. This could be solved by increasing the border width, but we also don't want to connect non-connected instances by setting the border width too large.

4.1.3.3 Comparing Pixel Occlusion to Bounding Box Occlusion

The impact of using the bounding box metric compared to the pixel boundary metric can be found in Figure 4.7 for the COCO dataset and in Figure 4.8 for the VOC dataset (these datasets will be explored in greater detail in Section 4.2). In these plots, we can see that the Bbox approach often has a significantly higher occlusion value than the pixel boundary; this is expected, as the bbox metric is a very forgiving

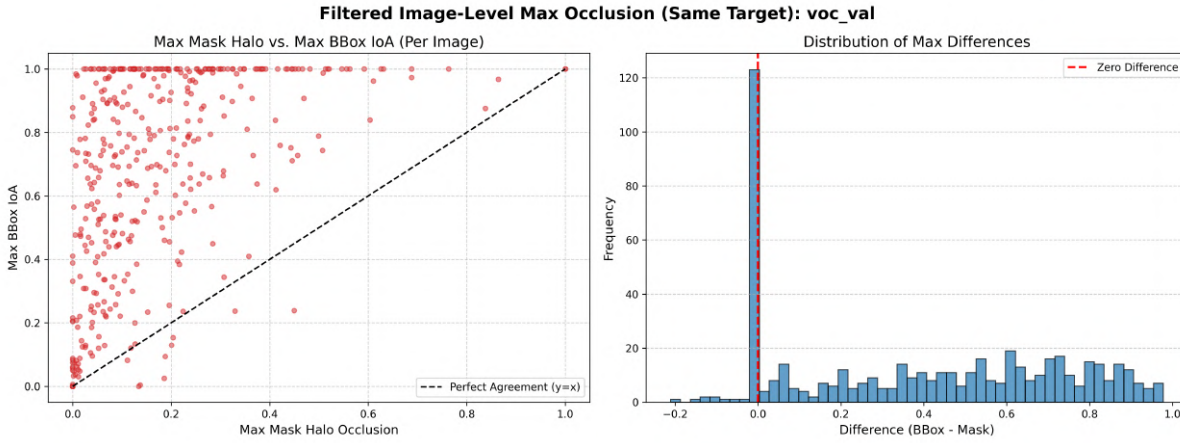


Figure 4.8: Figure showing the impact of using the bounding box metric compared to the pixel boundary metric on the VOC dataset to measure the occlusion for images.

heuristic compared to the overlapping boundary approach. It can also be observed that many images have a score of 1 for a bounding box but not for the pixel boundary; these are images where bounding boxes overlap, but the instances only partially match. Some examples are Figure 4.4, 4.3. When looking more in depth at the right plots in the COCO and VOC figures, it's notable that there are significantly more images with a difference of 0 than with any other value. This may not appear in the left scatterplot because most images in the datasets don't overlap, so these images are represented by many points at the same (0,0) location.

4.2 Datasets

4.2.1 CUB-200-2011

The CUB-200-2011 dataset is an extended version of the CUB-200 dataset [40]. This dataset contains 11,788 images of birds from 200 species. Every image is annotated with a label, one bounding box, and 15 Part Locations, thus a pixel mask indicating where the beak, head, etc., are. Combining these yields the complete bird mask. Given a fully annotated and extended dataset of images with only one bird instance per image, making this dataset perfect for WSOL but not for testing multi-instance segmentation. In Figure 4.9, you can find some example images and the given annotations

4.2.2 VOC 2012

The Pascal VOC 2012 dataset is a multi-class, multi-instance, fully annotated dataset. The dataset contains 9993 images, of which 7062 are annotated with segmentations and the others with only bounding boxes. It has 20 different classes, and sometimes multiple classes or the same class appear in a single image. As a result, we can use the dataset to evaluate multi-instance segmentation in Figure 4.11. You can find a co-occurrence matrix showing how often certain classes occur in the same image (the diagonal represents how often a class occurs at least twice in the same image). As you can see, the person class occurs more often with other classes than any other class type, and most images are single-class, which can lead to incorrect impressions of scores and performance when validating segmentation techniques. On average, the dataset contains 2.3 annotated objects per image.

4.2. DATASETS

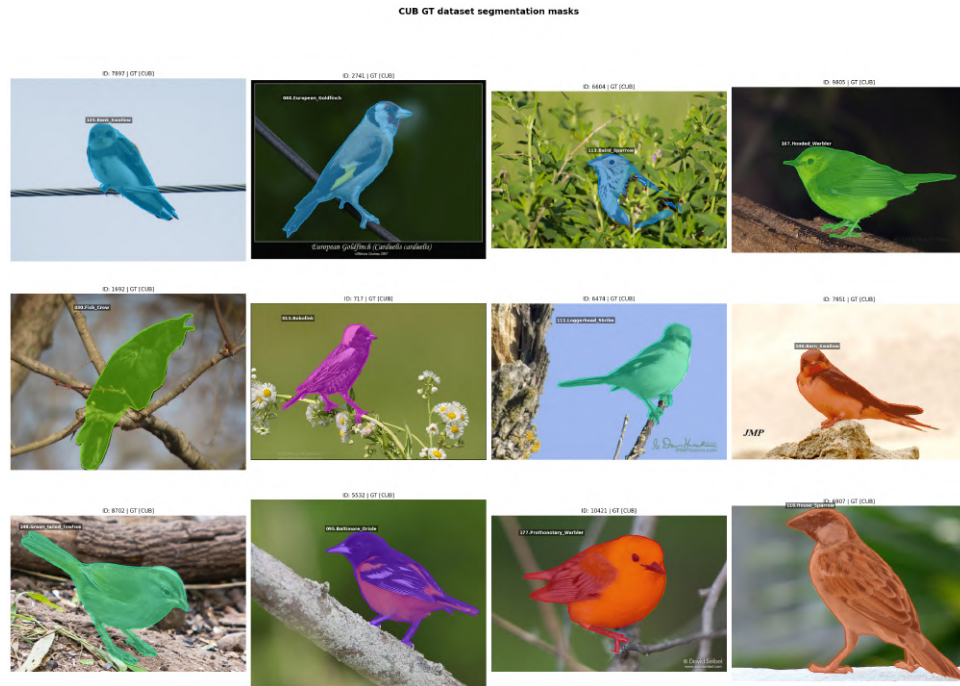


Figure 4.9: Randomly selected images with their segmentation mask from the CUB dataset

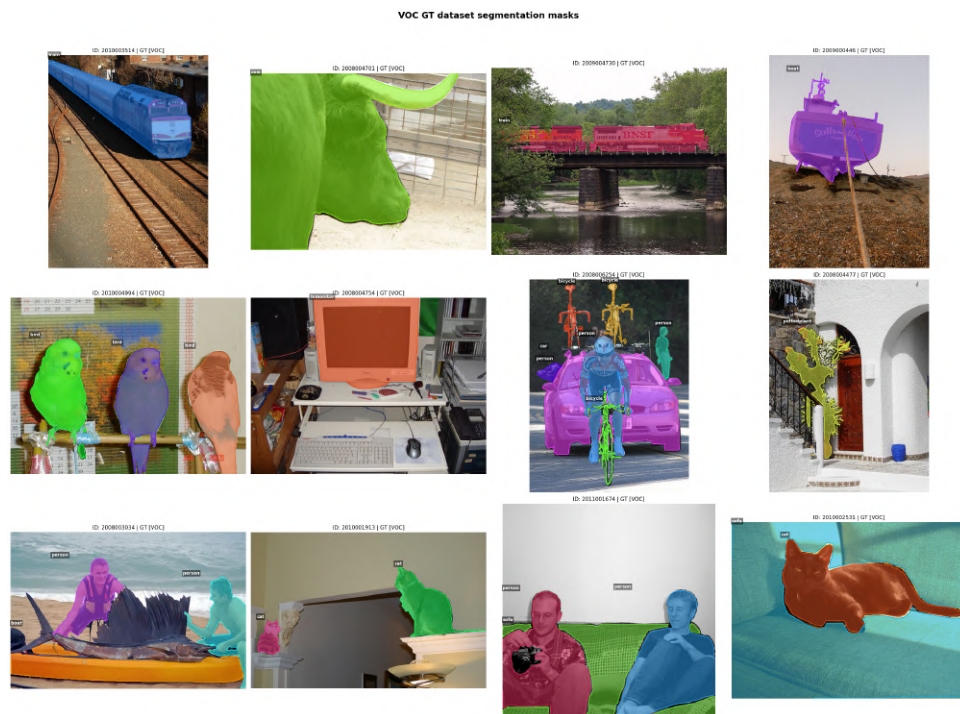


Figure 4.10: Randomly selected images with their segmentation mask from the VOC dataset

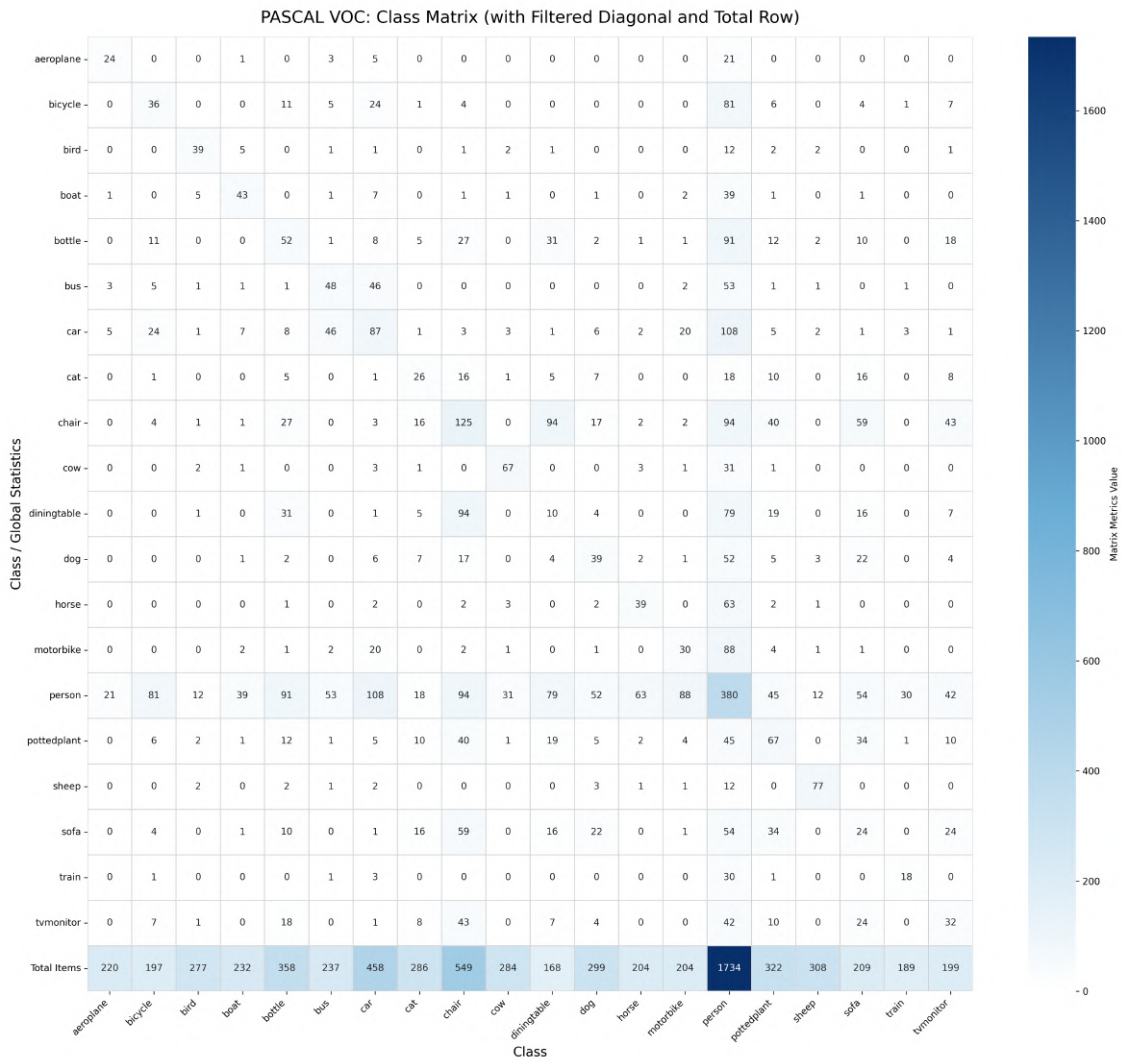


Figure 4.11: VOC dataset Co-occurrence matrix for every class, where the diagonal is the count if there are at least two instances of the same class

4.2. DATASETS

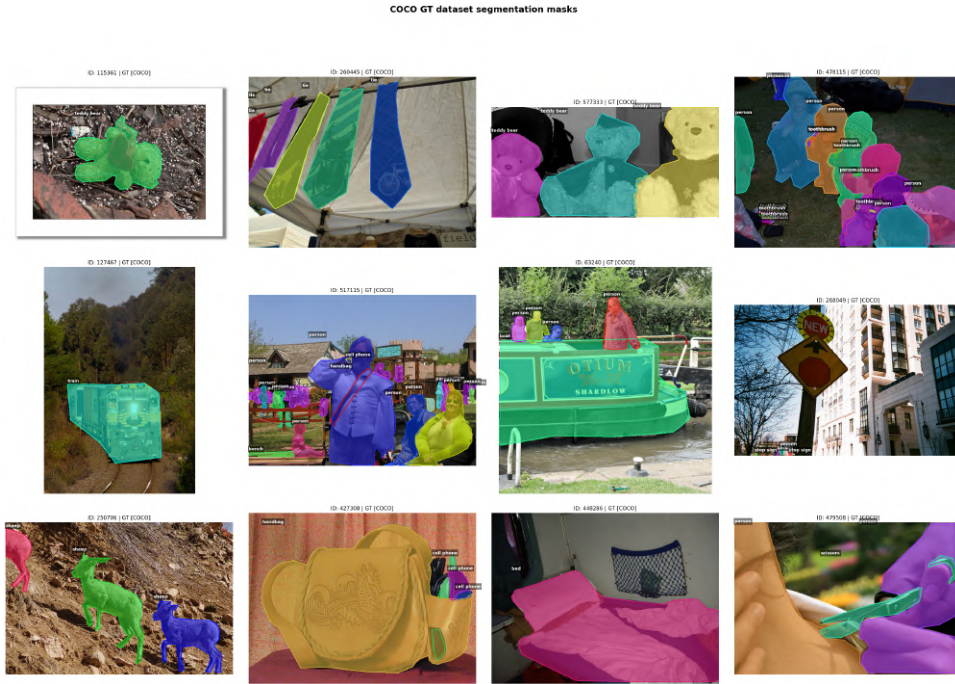


Figure 4.12: Randomly selected images with GT segmentation mask from the COCO dataset

4.2.2.1 MS COCO

MS COCO [26] is the largest of the datasets discussed, with over 328,000 labelled images and 91 object categories (82 of which have more than 5,000 labelled instances). This is a similar but significantly larger dataset than the VOC 2012 dataset. The larger number of object classes also increases the number of annotated instances per image, which is, on average, 7.7 for the MS COCO dataset. The dataset contains 2,500,000 annotated instances. In Figure 4.13, we compare the relative frequency of classes and instances of the COCO and VOC datasets, and we can notice that this is similar, with an outlier for the Person class, where the outlier is bigger in the COCO dataset. To keep the Figure comprehensible, we opted to show only the intersection of classes between VOC and COCO, not all COCO classes; Person is the only class that is an outlier.

4.2.3 Data Challenges

4.2.3.1 Dataset Instance Occlusion

As already mentioned in Section 4.1.3.2, datasets are never perfect; in this case, the issue stemmed from small errors in the segmentation masks that affected the specific metric we use. It is also important to analyse the dataset for its specific use case. In the plots of Figure 4.14, it can be seen that a significant number of images have no overlapping instances at all, and that this can be easily achieved with normal WSSS rather than WSIS by splitting instances based on whether the masks are separated. This is why the BAS WSSS method with instance split was added. The plot shows that the amount of overlap between all classes, compared to no overlap, is significantly smaller for VOC (1270 images with no overlap and $2156 = 1696 + 375 + 85$ images with occlusion) and COCO (6460 images with no overlap and 29875 images with occlusion). This is different when analysing the datasets of occlusions of the same class type; here, the VOC dataset has almost double the number of no-occlusion images (2233) than occlusion images (1193), and for the COCO dataset, a bit more no-occlusion images (19068) than

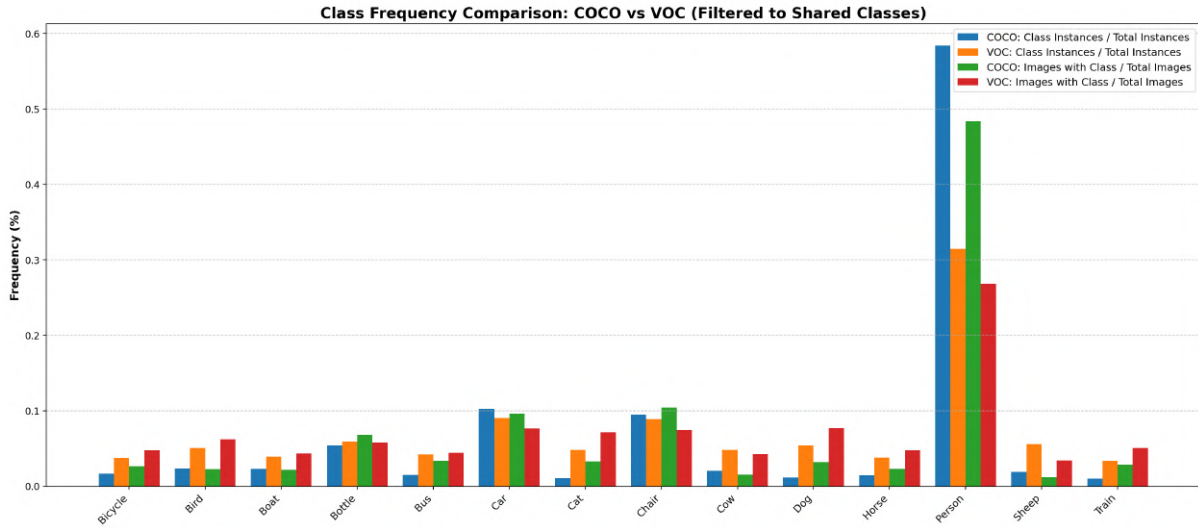


Figure 4.13: Comparison of the relative classes and relative instances in the VOC and COCO

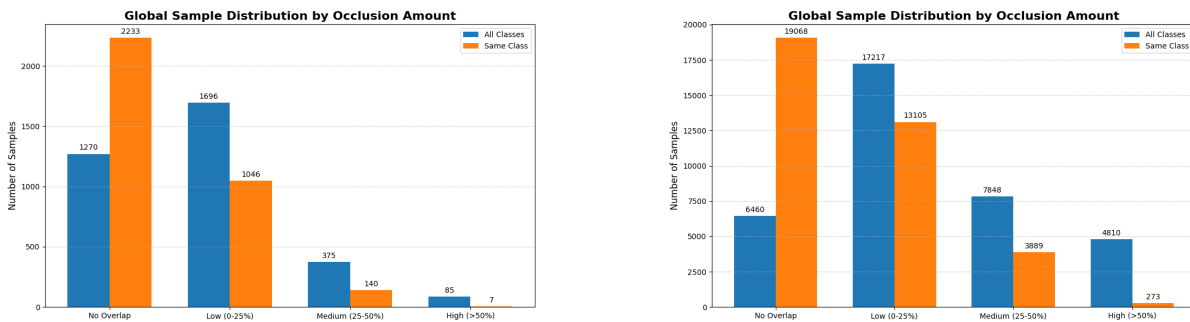


Figure 4.14: Left plot shows the number of images with certain amounts of occlusion for the VOC dataset; the right plot for the COCO.

occlusion images (17267). This is important to keep in mind, as the main challenge for WSIS is to segment touching instances of the same class. Thus, almost 50% of our dataset images do not contain these situations, but mAP scores are typically calculated across the entire dataset.

4.2.3.2 Handling Large Datasets

The VOC and COCO datasets are large, which is necessary for training the models, but they also pose challenges. Of course, training the models will take a long time, but handling these large datasets was sometimes more challenging than expected. There were two locations where models were trained and tested: my home computer, with limited storage, and the IMEC GPULAB, where storage is not limited in size but in the number of inodes, meaning it's limited in the number of files and folders. With the CUB and COCO datasets at around 50 GB in storage and more than 400,000 files, this seems reasonable, but this quickly escalates with the pipelines we have to run and their intermediate results. For Example, BAS has an intermediate step in which it stores results per image as .npy files, resulting in a 200+ GB folder. Moving this around is challenging, as it took more than a day to transfer it from my home PC to GPU LAB.

4.2.3.3 Dataset Splitting

In weakly supervised instance segmentation (WSIS), the model is trained on pseudolabels, so it might seem that we don't need to split the dataset. This is incorrect for several reasons: during inference, we provide only the image, not the class labels (which we do provide during training). It would also take too much time to retrain a model every time we want to do inference. Current research didn't use a 3-way split (train, validation, and test), so we also opted for a 2-way split (train and test) without a validation set; this allows us to compare results easily.

4.3 Analyse mAP score w.r.t Occlusion and instance counts

4.3.1 BAS (Background Activation Suppression)

As already discussed, BAS was originally a WSOL method and, in a later paper, was adapted to be used as a WSSS method; now it will be very primitively adapted to serve as a WSIS method. This is done by splitting masks of the same class when they do not touch. We can split this up into 3 interesting situations (see Figure 4.15):

- **True positive split:** the same class masks are correct and unconnected, and thus the instance separation happens correctly. This can be seen in the first row of Figure 4.15.
- **False negative split:** This happens when two separated instances are too close or overlapping; then the instances will not be split, resulting in two instances incorrectly being seen as one.
- **False positive split:** This happens when the same instance is completely obstructed by a different object, for Example, a pole or fence. This will incorrectly split the instance into two separate instances.

This method was added as a baseline to demonstrate how well a standard WSSS method can perform in a WSIS setting without employing novel or complex techniques. It will also show how different datasets

Cross-Dataset Model Instance Segmentation Comparison



Figure 4.15: These figures represent the most interesting cases where we split the instance masks based on. The first column shows the image without anything, the second column shows the ground-truth segmentation masks, and the third column shows the segmentation masks split by whether the masks are connected. All these images come from the COCO dataset and show only the masks for the Person class.

and metrics can influence the results.

4.3.1.1 Evaluation and Training

To evaluate the results of BAS for VOC, we used the provided models from their GitHub. Unfortunately, they never used the COCO dataset to test their method, so we had to modify the code to work with it. This code can be found here https://github.com/SteinVandenbroeke/thesis_BAS-Extension. We added a custom dataloader that was a drop-in replacement for the VOC dataloader.

4.3.1.2 Result analysis VOC dataset

As already mentioned, the first goal of the thesis was to develop an instance segmentation method. However, since we had already identified existing methods, we opted to identify possible flaws in them. The first thing that came to mind was measuring mAP scores with respect to instance occlusion. This can be found for the VOC dataset in Figure 4.18 and 4.19. The blue bars represent the values in a WSSS context; thus, the BAS masks are used without splitting disconnected masks, and the orange bars are the WSIS mAP values, where separated masks of the same class are split into multiple instances. A visual example can be found in Figure 4.20. Here, the middle image shows the masks split into two instances (WSIS), and the right image shows the case where we use the BAS WSSS mask without splitting. Here, it can be seen that the IoU scores for the sheep in the split image are 0.91 and 0.86, whereas in the non-split image they are only 0.54, because only one sheep is matched to the complete predicted mask for two sheep. Thus, as expected, the mAP values of the not-split masks are almost always lower. An ideal situation would be for the difference in mAP values to be larger, because the goal is to assess how well the WSIS performs, and many images still maintain high mAP values in the WSSS setting. Another thing to notice is that, as expected, the mAP for the same instance occlusion is significantly lower than the occlusion between all classes. Except for the High occlusion category, we will ignore it because only 7 images are available for this group; see Figure 4.14. It might be unexpected that the mAP value for the same instance occlusion images is still higher than zero; this is the result of two situations:

- Even when two items are combined, the masks will still match one of the ground truth instances, giving it a higher than 0 IoU score. An example can be seen in Figure 4.21
- The scores are calculated per image, and these other instances on this image with high occlusion can still have a high score, as all other non-occluding instances

In Figure 4.20, some generated example images are shown.

4.3.1.3 Result analysis COCO dataset

As already mentioned, this was not performed in the original paper, so we had to train our own model using modified code to work with the COCO dataset. The results were plotted in the same way as the VOC results (See Section 4.3.1.2) and can be found in Figures 4.22 and 4.23. The mAP scores are significantly lower than the VOC dataset; multiple reasons cause this: There are more classes and more complex classes, and there are also, on average, significantly more instances per image (see Section 4.2, this can also be seen in the generated example images in Figure 4.17); lastly, the COCO dataset is large and takes a long time to train finetuning parameters and hyperparameters is thus a complicated task where we only had limited time (training the model took easily 12 hours). Besides the overall lower mAP score, we see the same trend as in the VOC dataset and even more clearly. While the mAP



Figure 4.16: Example images generated with BAS for the VOC dataset. The left image shows the GT, and the right image shows the prediction.

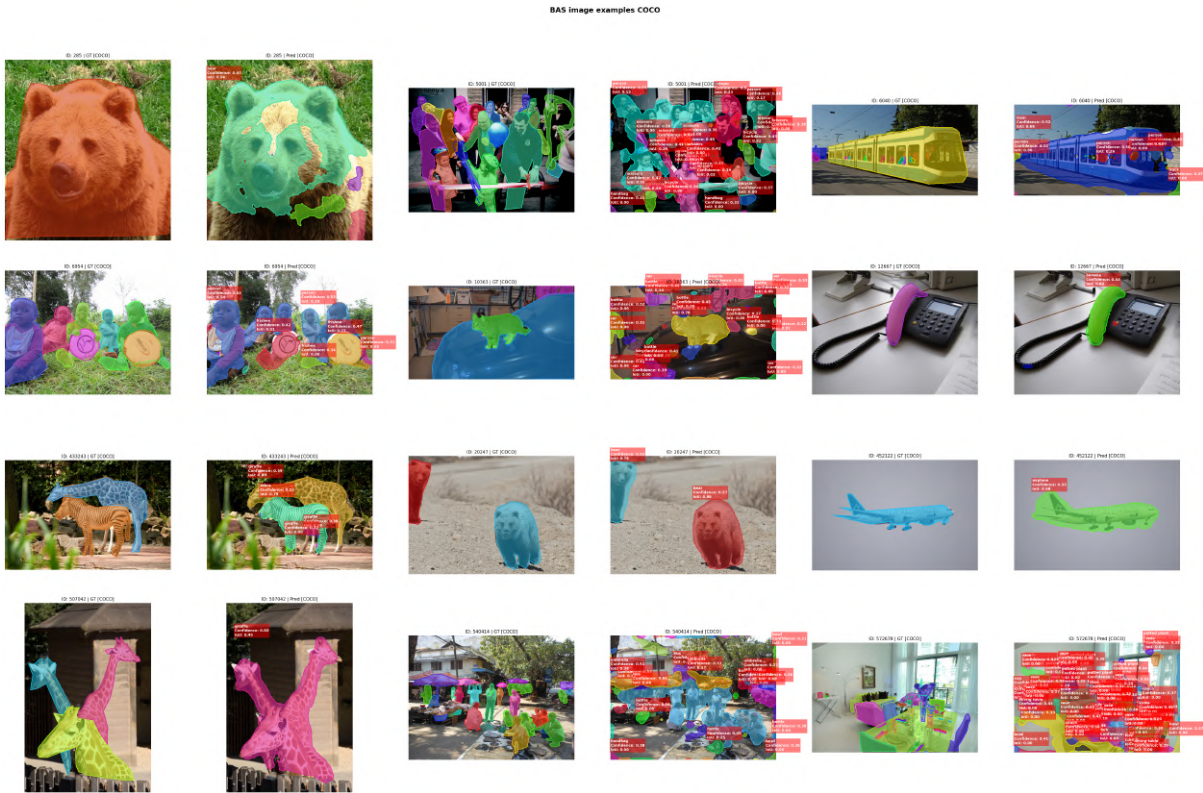


Figure 4.17: Example images generated with BAS for the COCO dataset. The left image shows the GT, and the right image shows the prediction.

4.3. ANALYSE MAP SCORE W.R.T OCCLUSION AND INSTANCE COUNTS

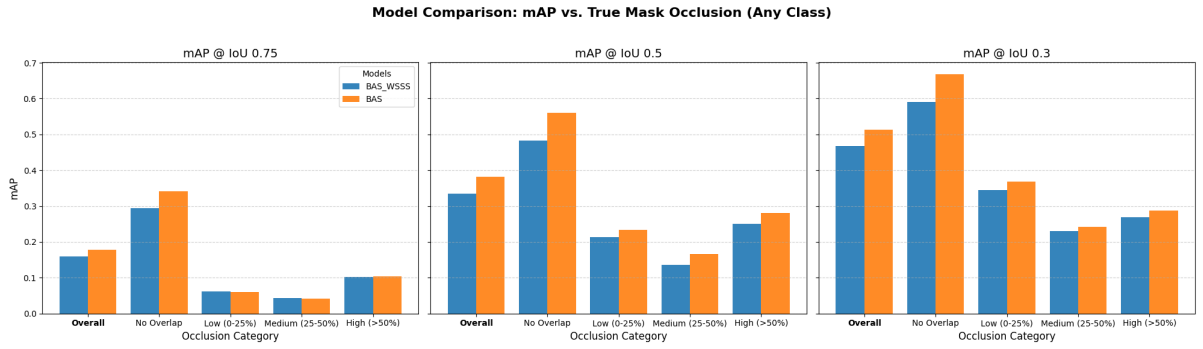


Figure 4.18: The mAP scores for the VOC dataset on the BAS method where the occlusion scores are calculated over all classes. BAS_WSSS values (blue bars) are the mAP scores for the images where the same class masks are never split even if the instance masks are completely separated; the BAS values (orange bars) are the values where the instance masks are separated if they are not connected.

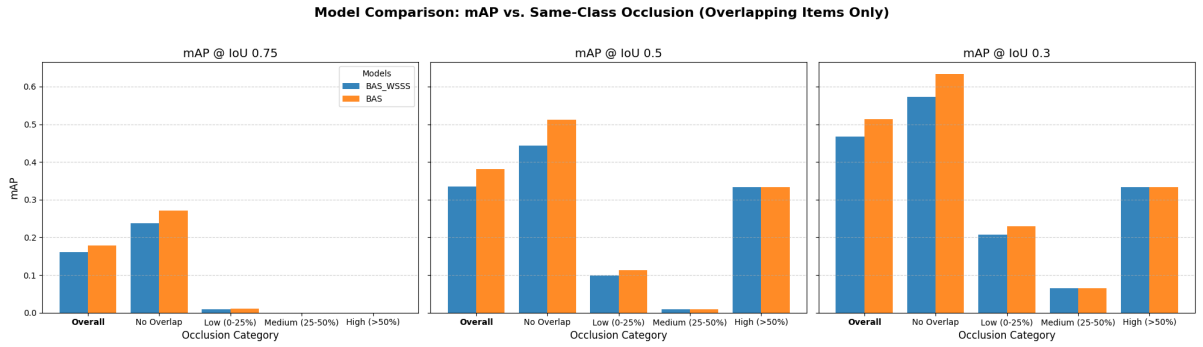


Figure 4.19: The mAP scores for the VOC where the occlusion scores are calculated over similar instances, meaning occlusion is only measured between the same class. BAS_WSSS values (blue bars) are the mAP scores for the images where the same-class masks are never split, even if the instance masks are completely separated; the BAS values (orange bars) are the values where the instance masks are separated if they are not connected.



Figure 4.20: Example image from the VOC dataset with the left image the gt, the middle image the WSIS prediction of BAS, and the right image the WSSS prediction of BAS.

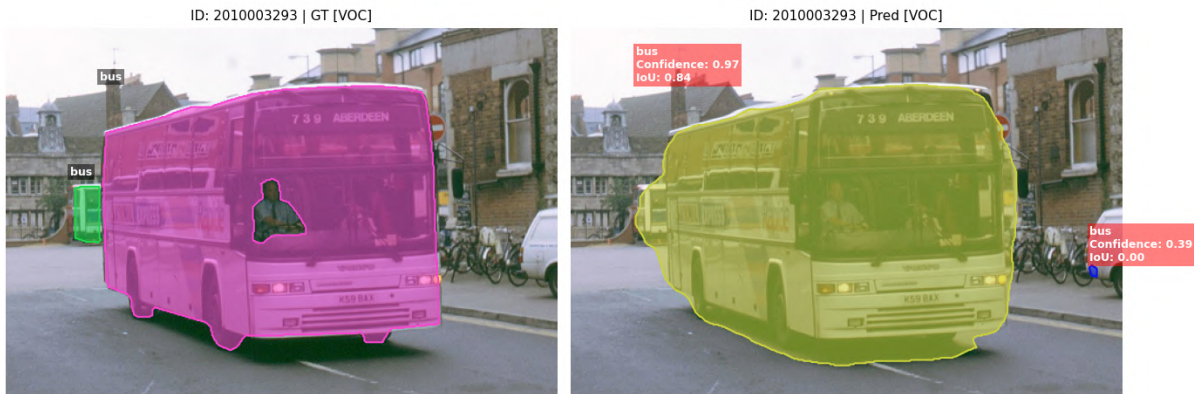
BAS VOC Combined by occlusion score

Figure 4.21: Example image where the mask of two buses is combined because of occlusion, but the IoU score for one bus is still high because the other bus is small.

results drop gradually as occlusion increases across all classes, they drop significantly when occlusion is measured between the same classes. This is expected, as there is no correct splitting between two instances of the same class when they are adjacent.

4.3.2 BESTIE (Beyond Semantic to Instance Segmentation)

As already explained in Section 3.4.2, BESTIE is created to solve the WSIS task. The method was tested in its paper [20] on the VOC and COCO datasets.

4.3.2.1 Challenges

Their code is available on GitHub (my modified fork https://github.com/SteinVandenbroeke/thesis_BAS-Extension); unfortunately, the shared code and models are only for the VOC dataset, so while the paper includes results for the COCO dataset, there is no easy way to reproduce them. Many open GitHub issues are requesting the code or models used in the paper; these have never received a response. Thus, we had to develop our own code and train a model on the COCO dataset to evaluate it in the same way as our other methods. This was a bigger-than-anticipated challenge because of the complicated, many-step training pipeline; there were many steps where something could go wrong, and the extremely long training time on the dataset and the large storage requirements for intermediate results did not help. And after many weeks of fixing the code, debugging, and training, I unfortunately had to stop due to time constraints. While most of the pipeline went well, it seems to have gone wrong in the final step of training the model. As already explained, the method used PAM (Peak Attention Module) in combination with WSSS maps. First, we retrained the PAM; this seemed to work well based on the intermediate results (see Figure 4.24). The WSSS maps of BAS were used because they had already been generated for the previous section, and this WSSS generation method performed well in the extended paper. After many hours of training and weeks of debugging, we still could not get the method working. This is unfortunate because the previous BAS method clearly showed that the mAP value is affected more by the COCO dataset than by the VOC dataset. This is also confirmed by the paper's mAP values, which drop significantly on the COCO dataset: instead of the VOC mAP@50 score of 0.41, the COCO mAP@50 drops to 0.28.

4.3. ANALYSE MAP SCORE W.R.T OCCLUSION AND INSTANCE COUNTS

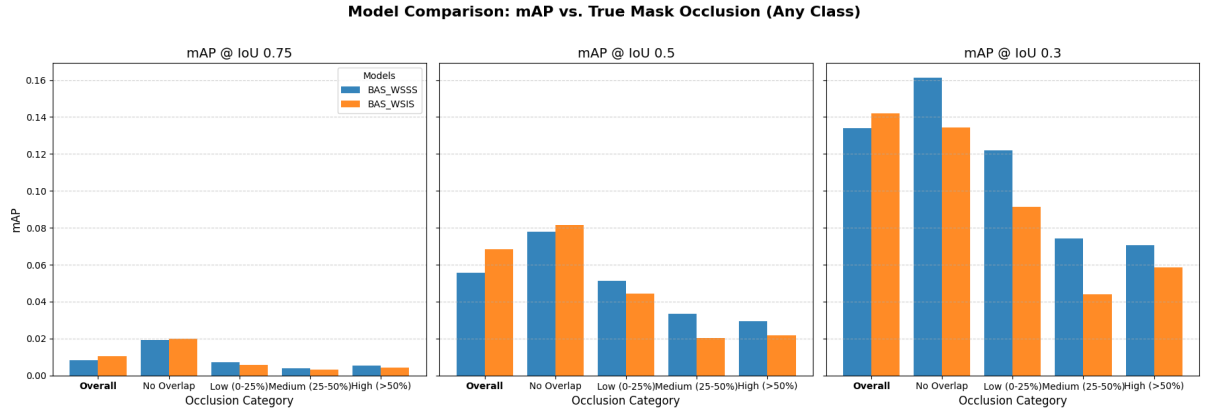


Figure 4.22: The mAP scores for the COCO dataset on the BAS method where the occlusion scores are calculated over all classes. BAS_WSSS values (blue bars) are the mAP scores for the images where the same class masks are never split even if the instance masks are completely separated; the BAS values (orange bars) are the values where the instance masks are separated if they are not connected.

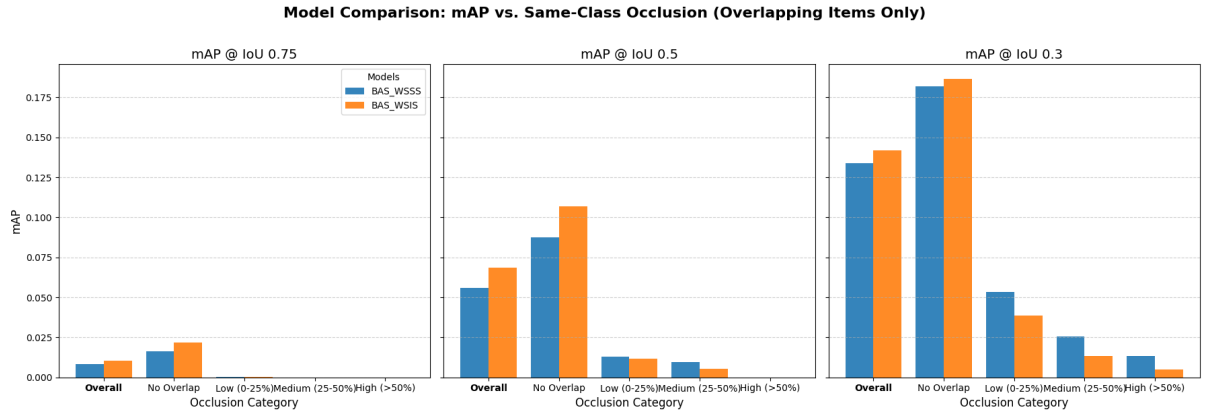


Figure 4.23: The mAP scores for the COCO dataset on the BAS method where the occlusion scores are calculated over similar instances, meaning occlusion is only measured between the same class. BAS_WSSS values (blue bars) are the mAP scores for the images where the same-class masks are never split, even if the instance masks are completely separated; the BAS values (orange bars) are the values where the instance masks are separated if they are not connected.



Figure 4.24: BESTIE PAM COCO results

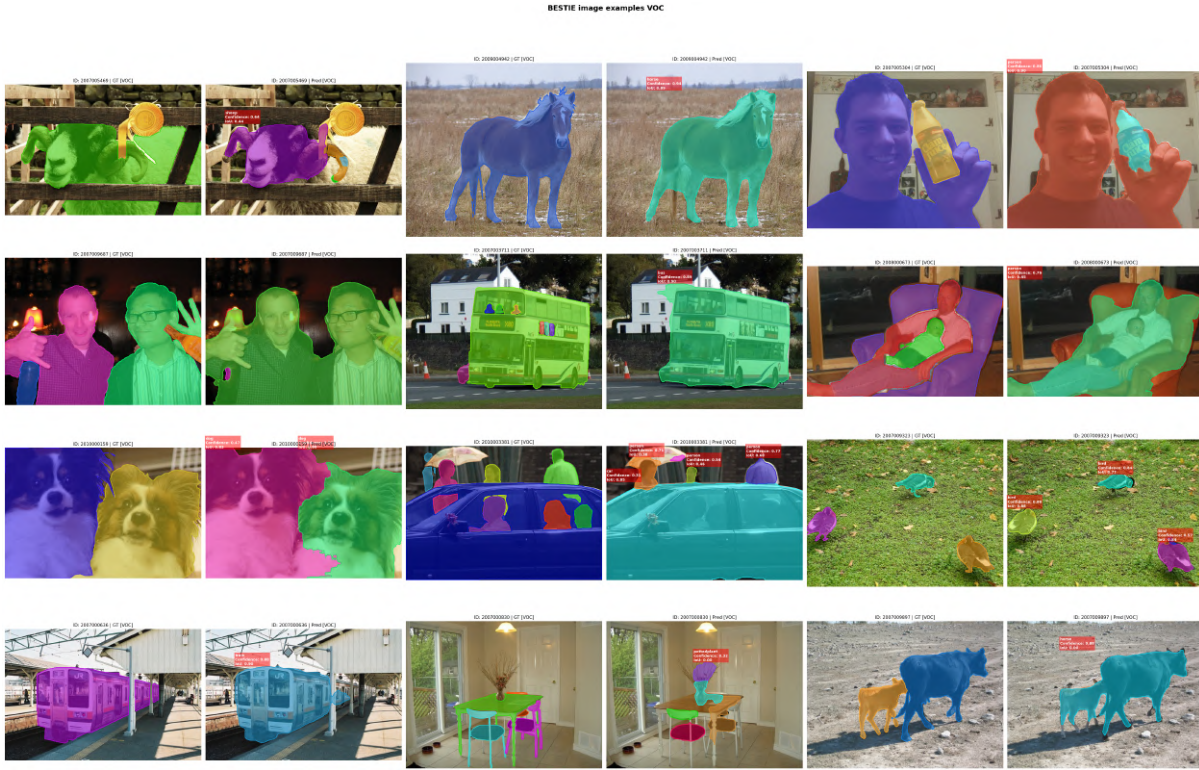


Figure 4.25: Example images generated with BESTIE for the VOC dataset. The left image shows the GT, and the right image shows the prediction.

4.3.2.2 Result Analysis VOC Dataset

Luckily, the codebase and the trained models, WSSS and Peak_points, generated by the PAM module were all available on their GitHub; this meant we only had to download the models and could then use them for inference on the VOC test set. This inference produced individual masks for the VOC dataset, allowing us to calculate mAP scores with respect to the occlusion of the GT instances. First, we can verify our run by comparing it to the paper’s mAP score; our mAP@50 is 0.416, while the paper reports 0.418. The difference is small enough to assume our inference and evaluation script is correct. As we can see here, the mAP score is similar to the previous reference method BESTIE, and also drops significantly for occlusion (any class and same class). This is not ideal, as the goal of this BESTIE method is to perform instance segmentation, which also splits instances of the same class. In Section 4.3.4, we will compare all methods. The reason it still drops significantly is that, when we look at the actual results, it sometimes succeeds in splitting the same instances of the same class and, of course, different classes, but it still often fails. This can be seen in the example image overview in Figure 4.25. The two dogs in row 3 column 2 are almost perfectly instance-segmented, but the cows in row 4 column 6 are combined into a single instance mask. When examining the generated masks for the test set, it is notable that segmentation fails more often than it succeeds. This is also confirmed by the plots in Figure 4.28, where we can see that the predicted instance count is way more often lower than the actual ground truth prediction count for the images with occlusion. We added Plot 2 as a reference; it shows how it should look if the approach works ideally: most of the images lie on the diagonal, and the spread above and below it is similar. The other plots clearly have fewer predicted instances than GT instances.

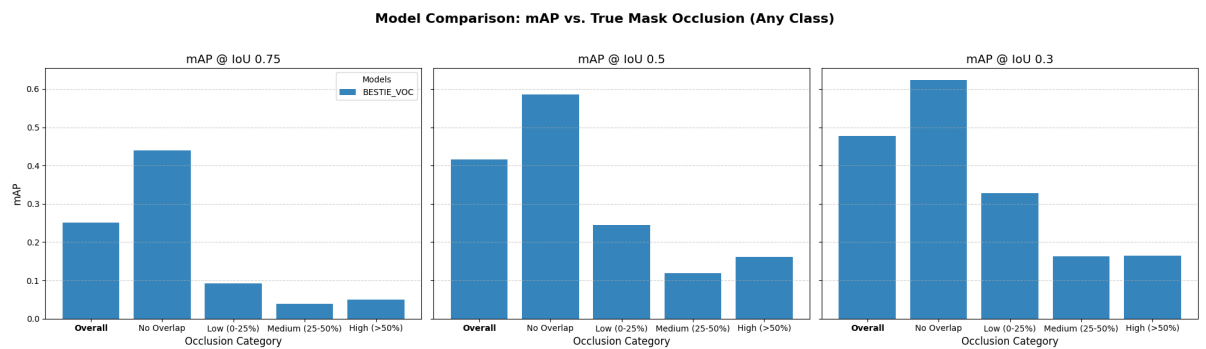


Figure 4.26: The mAP scores for the VOC dataset on the BESTIE method where the occlusion scores are calculated over all classes.

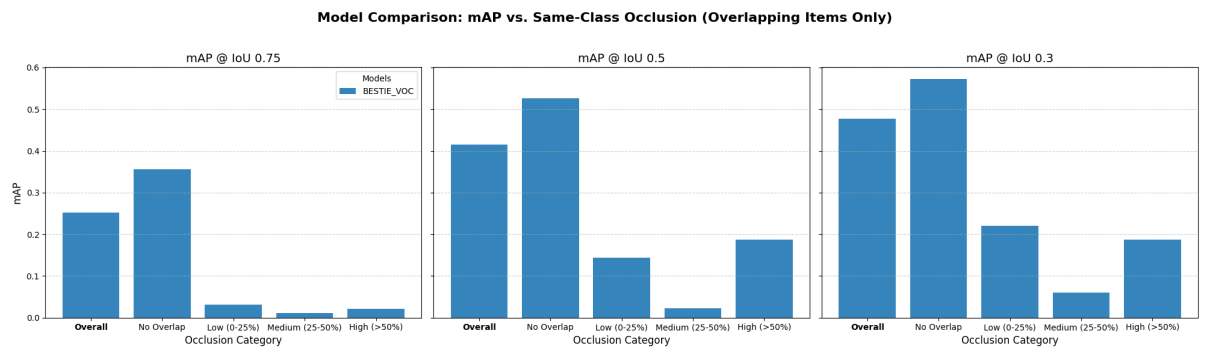


Figure 4.27: The mAP scores for the VOC dataset on the BESTIE method where the occlusion scores are calculated over similar instances, meaning occlusion is only measured between the same class.

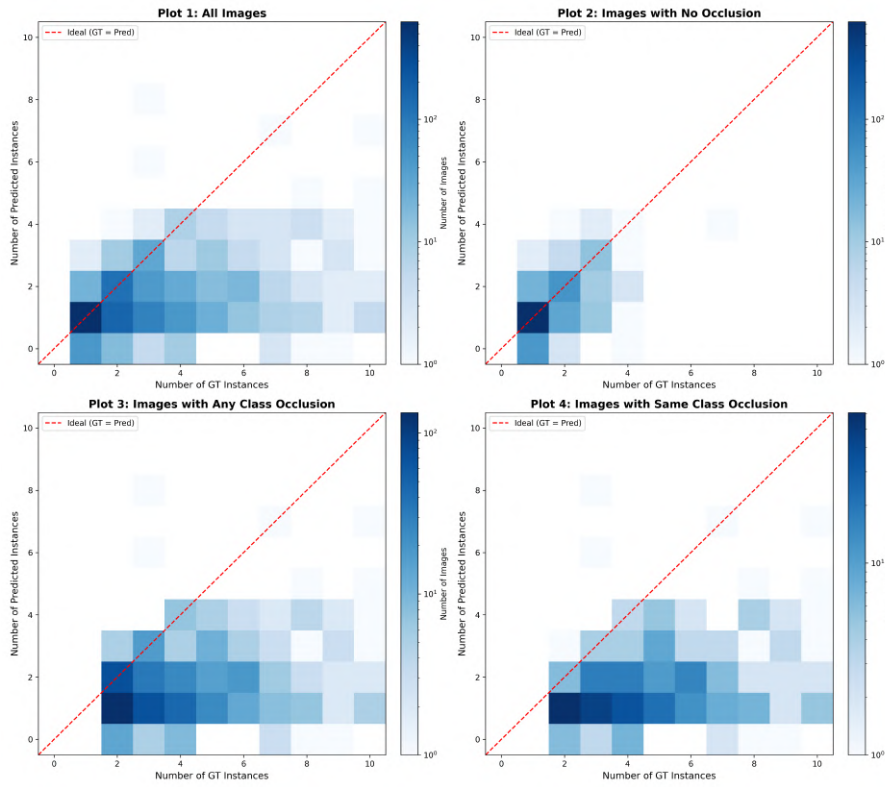


Figure 4.28: Heatmap plots that show the number of images that have a certain number of GT instances and the predicted instances. The results are cut off at 10 instances; above 10, most are 0 or 1 image. Meaning of the heatmap plots: under the diagonal means fewer instances predicted than GT; above the diagonal means more instances predicted than GT; on the diagonal means the number of predicted and GT instances is the same.

4.3.3 CIM (Complete Instance Mining)

Fortunately for us, the last WSIS method had all the code working on their GitHub (my modified GitHub fork https://github.com/SteinVandenbroeke/thesis_CIM), and the only challenge in running it was using a deprecated Python version on the IMEC servers with CUDA. They also provided their test and validation masks for the VOC and COCO datasets. Something that should become a standard in research, as it enables easier follow-up research and ensures the full code is shared, facilitating easy peer review.

4.3.3.1 Result Analysis VOC Dataset

Again, we can see the mAP scores on the dataset drop significantly as occlusion increases (See Figures 4.34a and 4.34b), as in the previous methods (see Comparison in Section 4.3.4), but to a lesser extent. And again, the value drops more for the same-class occlusion in comparison to any-class occlusion, as expected. When going through the dataset manually, we see far more successful instance splits; this is also visible in Figure 4.29, where it succeeds multiple times in splitting neighbouring items of the same class. There are still some images where it goes wrong, for Example in Figure 4.32. The better-looking results obtained by manually inspecting the generated mask are also confirmed by a heatmap showing the counts of generated and GT instances, with the results clustered around the diagonal, indicating that the number of predicted instances is similar to the number of GT instances.

4.3.3.2 Result Analysis COCO Dataset

Similar to previous methods, the COCO dataset is significantly more difficult for the CIM method, and thus the COCO mAP drops significantly compared to the VOC dataset in Figures 4.34d and 4.34d. And in Figure 4.33, the GT instance count is higher than the prediction instance count.

4.3.4 Comparing the Methods

Finally, we will compare several tested methods based on their mAP scores and results. When examining the No Overlap images, the winning method varies with the exact mAP and occlusion values. This indicates that no method is significantly better than the others in a WSSS setting. This changes if we look at the other categories (overall and occlusion). If we ignore the High occlusion class in VOC (due to limited samples), CIM almost always outperforms the other methods, except for Any class occlusions at mAP@50 and mAP@30, where both the BAS WSSS and WSIS methods are better. When looking at Figure 4.14, we can see that there are also significantly fewer images available for this occlusion category. And thus, it's hard to draw any conclusions from this result. When looking at the COCO dataset Figures 4.34c and 4.34d, the BAS approach is dramatically outperformed by the CIM method, as expected. This indicates that the overall mAP score can show whether a certain method outperforms another method in a WSIS setting (because the overall mAP score for CIM is always higher or equal); the results between BESTIE and CIM come close for the VOC dataset, and the reported scores for the COCO dataset in their papers are in Table 4.1. Here, CIM also outperforms BESTIE on the COCO dataset, as expected. Even with CIM scoring overall better for VOC, these don't really show the impact of how much better it performs compared to the CIM method when manually going through the dataset (See Figures 4.25 and 4.29) and looking at the predicted and GT counts of instances (See Figures 4.28 and 4.31); it performs significantly better. This seems better reflected in the Same Class mAP scores, where CIM scores significantly better. This is less of a problem with the COCO dataset, as it has more



Figure 4.29: Example images generated with BAS for the VOC dataset. The left image shows the GT, and the right image shows the prediction.

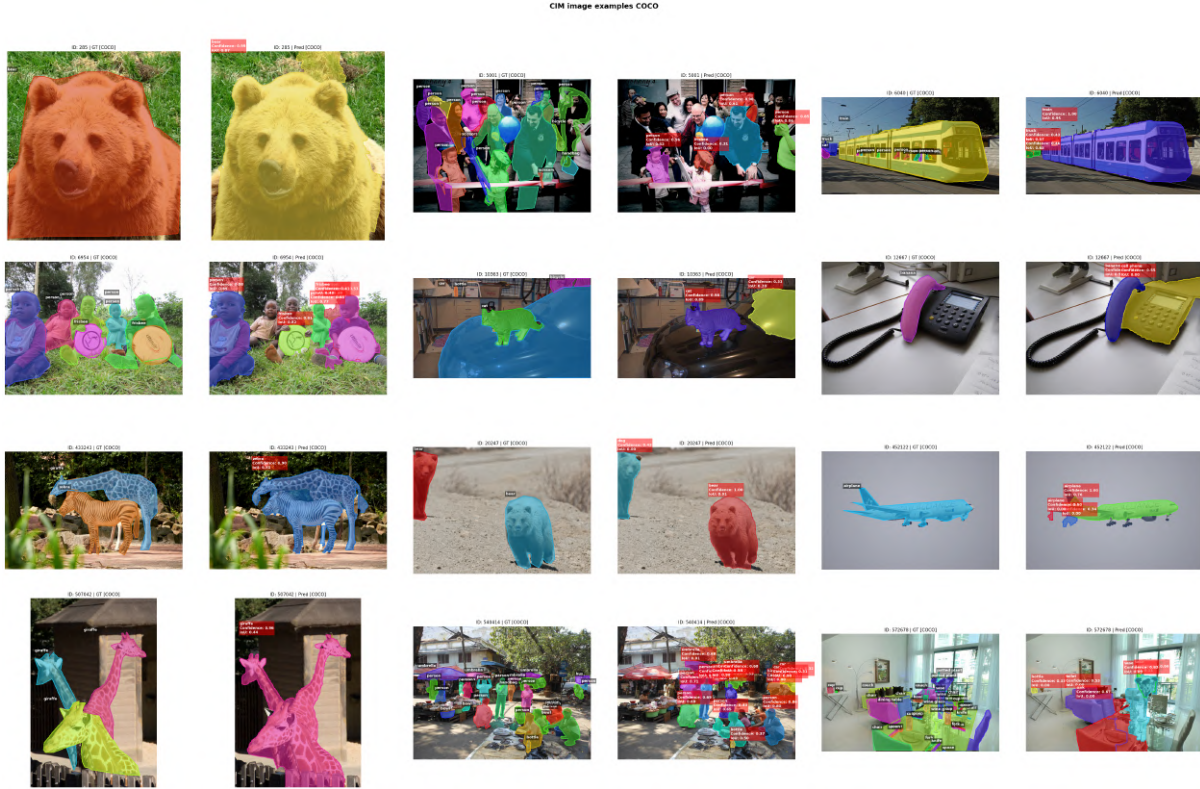


Figure 4.30: Example images generated with BAS for the COCO dataset. The left image shows the GT, and the right image shows the prediction.

4.3. ANALYSE MAP SCORE W.R.T OCCLUSION AND INSTANCE COUNTS

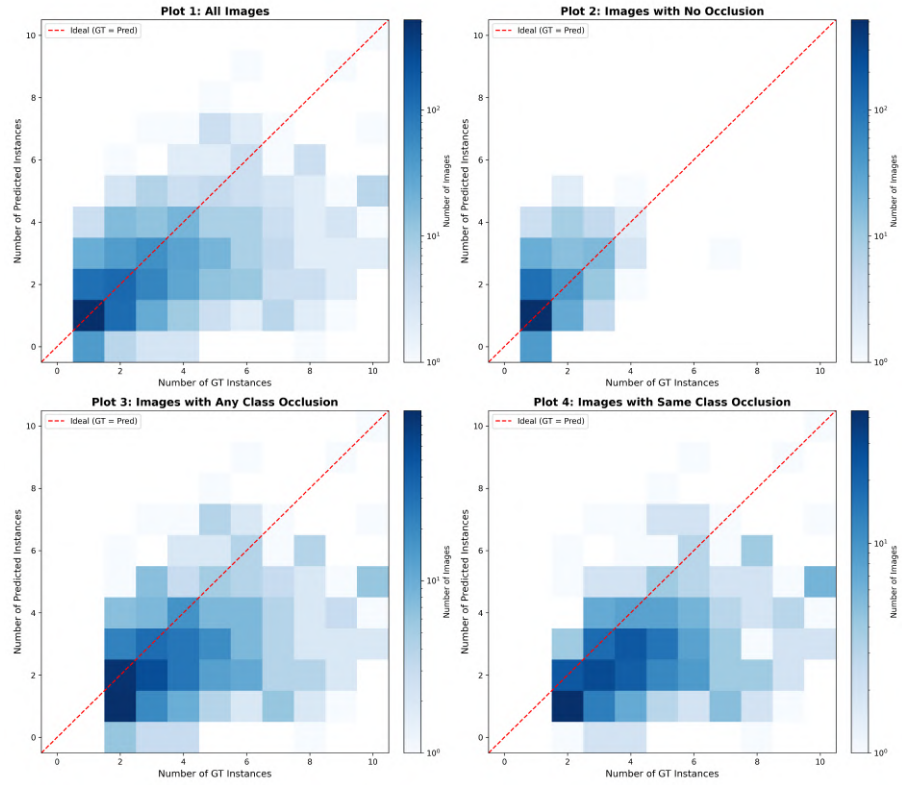


Figure 4.31: Heatmap for method CIM dataset VOC plots that show the number of images that have a certain number of GT instances and the predicted instances. The results are cut off at 10 instances; above 10, most are 0 or 1 image. Meaning of the heatmap plots: under the diagonal means fewer instances predicted than GT; above the diagonal means more instances predicted than GT; on the diagonal means the number of predicted and GT instances is the same.

CIM VOC Failed Instance Split



Figure 4.32: CIM method VOC dataset example of failed instance splitting.

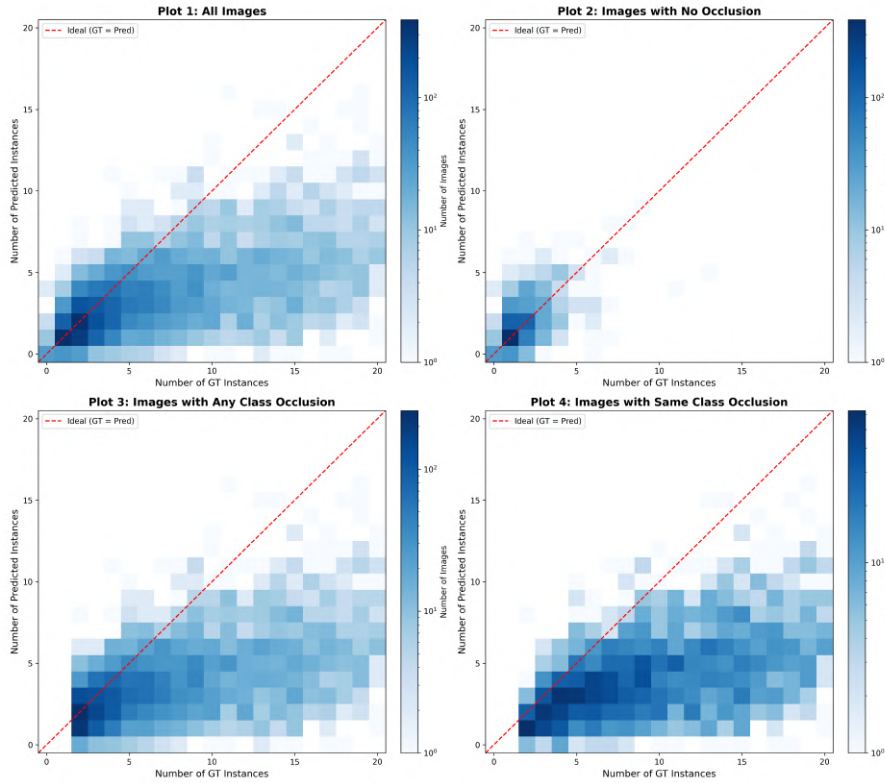


Figure 4.33: Heatmap for method CIM dataset COCO plots that show the number of images that have a certain number of GT instances and the predicted instances. The results are cut off at 10 instances; above 10, most are 0 or 1 image. Meaning of the heatmap plots: under the diagonal means fewer instances predicted than GT; above the diagonal means more instances predicted than GT; on the diagonal means the number of predicted and GT instances is the same.

4.3. ANALYSE MAP SCORE W.R.T OCCLUSION AND INSTANCE COUNTS



Figure 4.34: The mAP scores for the COCO dataset on the CIM method. Graphs (a) and (c) show occlusion scores calculated over all classes, while (b) and (d) show scores calculated over similar instances (occlusion measured only between the same class).

images with occlusion within the same class. Unfortunately, there was not enough time to test additional methods and get every dataset working, so the results are too limited to draw definitive conclusions.

Method	AP	mAP@50	mAP@75
BESTIE	14.4	28.0	13.5
CIM	17.2	29.7	17.3

Table 4.1: Reported mAP scores of papers

Condition	IoU	Method	Overall	No Overlap	Low (0-25%)	Medium (25-50%)	High (>50%)
Any Class	0.75	CIM	24.9	38.1	13.6	6.1	10.4
		BESTIE	25.1	44.0	9.2	3.8	5.0
		BAS-WSSS	16.0	29.3	6.2	4.3	10.2
		BAS-WSIS	17.8	34.1	6.1	4.2	10.4
	0.50	CIM	46.3	58.5	33.3	14.1	19.9
		BESTIE	41.6	58.5	24.4	11.9	16.1
		BAS-WSSS	33.5	48.2	21.4	13.5	25.0
		BAS-WSIS	38.1	56.0	23.4	16.7	28.1
	0.30	CIM	54.2	64.0	43.6	17.2	20.1
		BESTIE	47.7	62.3	32.8	16.2	16.5
		BAS-WSSS	46.8	59.1	34.5	23.0	26.9
		BAS-WSIS	51.3	66.8	36.8	24.2	28.8
	0.75	CIM	24.9	31.8	9.6	3.8	4.9
		BESTIE	25.1	35.6	3.1	1.1	2.1
		BAS-WSSS	16.0	23.8	0.9	0.0	0.0
		BAS-WSIS	17.8	27.2	1.0	0.0	0.0
Same Class	0.50	CIM	46.3	54.4	22.8	8.9	21.5
		BESTIE	41.6	52.6	14.3	2.3	18.8
		BAS-WSSS	33.5	44.4	9.9	1.0	33.3
		BAS-WSIS	38.1	51.2	11.3	0.9	33.3
	0.30	CIM	54.2	61.0	32.7	10.2	21.5
		BESTIE	47.7	57.2	22.0	6.0	18.8
		BAS-WSSS	46.8	57.3	20.7	6.6	33.3
		BAS-WSIS	51.3	63.3	23.0	6.5	33.3

Table 4.2: Performance comparison of models on the VOC dataset across different occlusion categories (AP scores). Bold numbers are the best-performing values.

4.3.5 Performance of WSIS Method on Single Instance Dataset

To test this, we decided to use BESTIE because it uses WSSS from another method; this allowed us to use the original BAS WSSS masks as input for BESTIE. In the ideal case, BESTIE doesn't change anything, and we receive the same masks and, thus, the same scores. In tables 4.4 and 4.5, as you can see, there are 3 available BESTIE methods; while the trained model stays the same, they change how the data is interpreted (see Figure 4.35):

- **BESTIE Best masks:** here, the evaluation is performed on the mask with the highest confidence score.
- **BESTIE All masks:** here the evaluation is done over all masks.
- **BESTIE Combined masks:** all prediction masks are combined into a single mask.

Because we are now measuring in a WSOL setting, we also opted to calculate our metrics using bounding boxes rather than the masks themselves, following the approach used in other WSOL papers. Over-

Condition	IoU	Method	Overall	No Overlap	Low (0-25%)	Medium (25-50%)	High (>50%)
Any Class	0.75	CIM	9.8	15.9	8.6	2.6	3.0
		BESTIE	X.X	X.X	X.X	X.X	X.X
		BAS-WSSS	0.8	1.9	0.7	0.4	0.5
		BAS-WSIS	1.0	2.0	0.6	0.3	0.4
	0.50	CIM	18.6	25.2	16.4	6.6	5.1
		BESTIE	X.X	X.X	X.X	X.X	X.X
		BAS-WSSS	5.6	7.8	5.1	3.3	2.9
		BAS-WSIS	6.8	8.1	4.4	2.0	2.2
	0.30	CIM	22.9	27.8	21.1	9.0	6.1
		BESTIE	X.X	X.X	X.X	X.X	X.X
		BAS-WSSS	13.4	16.1	12.2	7.4	7.0
		BAS-WSIS	14.2	13.4	9.1	4.4	5.8
Same Class	0.75	CIM	9.8	13.8	2.9	1.1	0.5
		BESTIE	X.X	X.X	X.X	X.X	X.X
		BAS-WSSS	0.8	1.6	0.0	0.0	0.0
		BAS-WSIS	1.0	2.2	0.1	0.0	0.0
	0.50	CIM	18.6	24.1	8.0	3.7	0.5
		BESTIE	X.X	X.X	X.X	X.X	X.X
		BAS-WSSS	5.6	8.8	1.3	0.9	0.0
		BAS-WSIS	6.8	10.7	1.2	0.5	0.0
	0.30	CIM	22.9	27.7	12.2	5.4	1.6
		BESTIE	X.X	X.X	X.X	X.X	X.X
		BAS-WSSS	13.4	18.2	5.4	2.6	1.4
		BAS-WSIS	14.2	18.6	3.8	1.4	0.5

Table 4.3: Performance comparison of models on the COCO dataset across different occlusion categories (AP scores). The Bestie results are empty because this method was not used on the COCO dataset. Bold numbers are the best-performing values.

all, the tables show that the BESTIE model, compared to the BAS model, has a negative impact on performance. When using the BESTIE method on top of the BAS method, it performs best in a combined-mask setting, but this makes it practically unusable. If we know we will combine all masks, we are better off not using a WSIS method. When using all masks, as it would be a practical setting because we want multiple instances, the scores drop dramatically, making BESTIE unusable. This is probably because the distinguishing features among birds are their subparts, which divide them. With single instances of very distinct classes, we don't have this problem because the VOC dataset never splits instances.

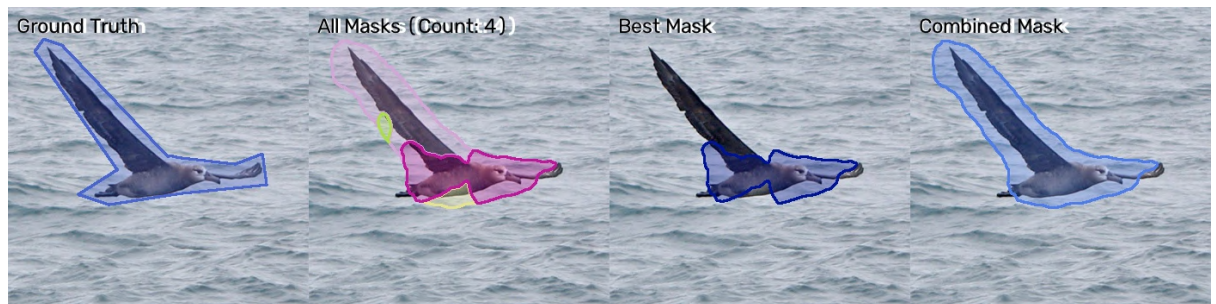


Figure 4.35: BESTIE image example with all mask combinations

Method	Top-1	Top-5	GT-known
BAS	72.6	85.5	90.7
BESTIE All masks	24.9	37.8	61.1
BESTIE Combined mask	25.4	25.4	79.3

Table 4.4: Localisation Metrics.

Method	IoU30	IoU50	IoU75
BAS WSOL	99.4	90.7	43.4
BESTIE Best masks	35.6	16.1	1.52
BESTIE All masks	30.8	19.6	2.00
BESTIE Combined masks	93.9	66.1	10.78

Table 4.5: IoU Metrics.

5 Conclusion

5.1 Answers to Research Questions

5.1.1 The Current State of WSIS

To assess the current state of WSIS methods, we reevaluated the methods BAS in WSSS setting, BAS in WSIS setting, BESTIE, and CIM. Where BAS in WSSS setting and BAS in WSIS setting were used to compare the impact (using the mAP metric) of the novel WSIS approaches (BESTIE and CIM). Here we can see that both novel WSIS methods delivered notable improvements in the original metric (as discussed in our results section, overall mAP) and when manually verifying the instance segmentation results.

5.1.2 WSOL Metrics for WSIS

Most WSIS papers report performance only using the mAP metric. We wanted to verify this metric by adding additional evaluation criteria: the amount of occlusion between two instances in an image and whether the instances were from the same class. Splitting this up showed, for the VOC dataset, a more closely aligned mAP score (same-class with overlap metric) with the model's actual perceived performance. To assess the actual perceived performance from the created masks, we plotted the number of GT instances against the number of predicted instances. For the COCO dataset, the gap in overall mAP (no overlap and any overlap) between CIM and BAS was much more closely aligned with perceived performance than in the VOC dataset (without the need for the overlap metric). This is probably because the COCO dataset is overall significantly more difficult than the VOC dataset, with on average more instances per image. Thus, simple, non-WSIS-specific cases cannot boost the overall mAP score. Thus, the conclusion is that the mAP score is a good metric for comparing different methods when datasets have enough challenging images that test actual instance segmentation. Overall, based on the current evidence, we would also recommend showing more examples of generated masks in papers and reporting mAP scores with respect to occlusion and same/different instances to make the evaluation more complete.

5.1.3 Instance Segmentation for WSOL

Finally, we tested whether the BESTIE WSIS method decreased the performance of the BAS WSOL method. To do this, we used the generated masks from the BAS method as input to the BESTIE method, since it relies on external WSSS masks. Then the peak regions were created. Then, instead of testing both methods with our default script, we tested them with a modified version that uses bounding boxes instead of masks to calculate the mAP, TOP-1, TOP-5, and GT-known scores; all of these showed a significant drop in performance for the BESTIE WSIS method compared to the BAS WSOL method. This was tested on only one dataset, and only two methods were compared. Thus, the overall conclusion is that WSIS methods **can** degrade performance compared to the default WSOL method. Considering

this, it is probably highly dependent on the dataset and the WSOL and WSIS methods used, as WSIS are always very novel and work very differently.

5.2 Limitations of the Work

The complex pipelines required for WSIS methods, combined with large datasets, missing open-source code, and deprecated Python dependencies, made training and evaluating these models significantly challenging. Tasks that initially appeared straightforward ultimately required substantial time and effort to troubleshoot. While prior coursework provided a strong foundation in training deep neural networks, this thesis highlighted the practical, often undocumented hurdles of adapting legacy codebases and managing massive datasets in a research environment. Consequently, these technical bottlenecks created a time shortage that limited the incorporation of additional methods and datasets, thereby restricting the generalizability of our claims. Nevertheless, this work provides valuable insights into novel evaluation techniques for instance segmentation and highlights critical areas for future WSIS research.

5.3 Future Work

5.3.1 Evaluating Additional WSIS Architectures

Future work should expand the number of WSIS methods, for example, by adding irNET, DiscoBox, BBAM, or newly created methods. Testing additional methods will help confirm whether the boundary-pixel occlusion metric proposed in this study is universally applicable and robust across diverse methods.

5.3.2 Closing the Gap with Supervised Learning

Closing the Gap with Supervised Learning: As WSIS is still a developing field, novel solutions should be developed to reduce the performance gap to fully supervised instance segmentation models. Investigating new unsupervised instance segmentation techniques to handle same-class instance segmentation better.

5.3.3 Completing the BESTIE COCO Evaluation

Due to the technical constraints and time limitations encountered during this work, the occlusion-based evaluation of the BESTIE framework on the COCO dataset was left incomplete. Successfully adapting the codebase and training BESTIE on COCO to measure its performance against the occlusion metric would provide a much-needed, comprehensive comparison against CIM and the BAS WSIS method baseline.

5.3.4 Expanding the WSOL Compatibility Analysis

Our experiment demonstrates that the BESTIE WSIS method decreased performance on the WSOL task (using the CUB dataset). Future studies should apply various WSIS methods across multiple single-instance localisation datasets to determine whether this performance drop is a fundamental characteristic

5.3. *FUTURE WORK*

of instance-splitting architectures or merely an artefact of the specific BESTIE and BAS WSSS pipeline used in this research.

Bibliography

- [1] A Beginner's Guide to Image Segmentation - WobblePic — wobblepic.com. <https://wobblepic.com/blog/beginners-guide-to-image-segmentation/>. [Accessed 21-08-2026]. 5
- [2] Image Segmentation: Techniques and Applications - GeeksforGeeks — geeksforgeeks.org. <https://www.geeksforgeeks.org/computer-vision/image-segmentation-techniques-and-applications/>. [Accessed 14-04-2026]. 1
- [3] Tutorial: Mechanical Turk Best Practices — news.findingfive.com. <https://news.findingfive.com/2020/05/17/mechanical-turk-best-practices/#{R}easonable-{P}ay>. [Accessed 21-08-2026]. 1
- [4] Sura Abdullah. Survey: Privacy-preserving in deep learning based on homomorphic encryption. 01 2022. 4
- [5] Jiwoon Ahn, Sunghyun Cho, and Suha Kwak. Weakly supervised learning of instance segmentation with inter-pixel relations. pages 2204–2213, 06 2019. 9, 13
- [6] Amy Bearman, Olga Russakovsky, Vittorio Ferrari, and Li Fei-Fei. What's the point: Semantic segmentation with point supervision. In *European conference on computer vision*, pages 549–565. Springer, 2016. 10, 11
- [7] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, Rishi Hazra, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollár, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. Sam 3: Segment anything with concepts, 2025. 1
- [8] Mattia Casini, Paolo De Angelis, Eliodoro Chiavazzo, and Luca Bergamasco. Current trends on the use of deep learning methods for image analysis in energy applications. *Energy and AI*, 15:100330, 01 2024. 6
- [9] Tao Chen, Yazhou Yao, Xingguo Huang, Zechao Li, Liqiang Nie, and Jinhui Tang. Spatial structure constraints for weakly supervised semantic segmentation. *IEEE Transactions on Image Processing*, 33:1136–1148, 2024. 9
- [10] Bowen Cheng, Omkar Parkhi, and Alexander Kirillov. Pointly-supervised instance segmentation. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2607–2616, 2022. 13
- [11] Tianheng Cheng, Xinggang Wang, Shaoyu Chen, Qian Zhang, and Wenyu Liu. Boxteacher: Exploring high-quality pseudo labels for weakly supervised instance segmentation. In *IEEE/CVF*

Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022. 13

- [12] Junsuk Choe, Seong Joon Oh, Seungho Lee, Sanghyuk Chun, Zeynep Akata, and Hyunjung Shim. Evaluating weakly supervised object localization methods right. pages 3130–3139, 06 2020. 9
- [13] Alexey Dosovitskiy and Thomas Brox. Inverting visual representations with convolutional networks. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4829–4837, 2016. 8
- [14] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The PASCAL Visual Object Classes Challenge 2012 (VOC2012) Results. <http://www.pascal-network.org/challenges/VOC/voc2012/workshop/index.html>. 1, 10
- [15] Mark Everingham, Luc Van Gool, Christopher Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88:303–338, 06 2010. 9
- [16] geeksforgeeks. Unsupervised Machine Learning - GeeksforGeeks — geeksforgeeks.org. <https://www.geeksforgeeks.org/machine-learning/unsupervised-learning/>, 2026. [Accessed 14-06-2026]. 9
- [17] Ross B. Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 580–587, 2013. 9
- [18] Felix Grün, Christian Rupprecht, Nassir Navab, and Federico Tombari. A taxonomy and library for visualizing learned features in convolutional neural networks. 06 2016. 7, 8
- [19] Zhizhong Huang, Mingliang Dai, Yi Zhang, Junping Zhang, and Hongming Shan. Point, segment and count: A generalized framework for object counting. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17067–17076, 2024. 1
- [20] Beomyoung Kim, Youngjoon Yoo, Chaeun Rhee, and Junmo Kim. Beyond semantic to instance segmentation: Weakly-supervised instance segmentation via semantic knowledge transfer and self-refinement, 2022. 9, 18, 36
- [21] Hyeokjun Kweon and Kuk-Jin Yoon. From sam to cams: Exploring segment anything model for weakly supervised semantic segmentation. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19499–19509, 2024. 9
- [22] Shiyi Lan, Zhiding Yu, Christopher Choy, Subhashree Radhakrishnan, Guilin Liu, Yuke Zhu, Larry S. Davis, and Anima Anandkumar. Discobox: Weakly supervised instance segmentation and semantic correspondence from box supervision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3406–3416, October 2021. 13
- [23] Jungbeom Lee, Jihun Yi, Chaehun Shin, and Sungroh Yoon. Bbam: Bounding box attribution map for weakly supervised semantic and instance segmentation. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2643–2651, 2021. 13
- [24] Yuhong Li, Xiaofan Zhang, and Deming Chen. Csnet: Dilated convolutional neural networks for understanding the highly congested scenes. 02 2018. 1
- [25] Zecheng Li, Zening Zeng, Yuqi Liang, and Jin-Gang Yu. Complete instances mining for weakly supervised instance segmentation. In *Proceedings of the Thirty-Second International Joint Con-*

- ference on Artificial Intelligence*, IJCAI-2023, page 1142–1150. International Joint Conferences on Artificial Intelligence Organization, August 2023. 9, 19, 20
- [26] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft coco: Common objects in context, 2015. 1, 10, 29
 - [27] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. *CoRR*, abs/1411.4038, 2014. 6
 - [28] Jonathan Long, Ning Zhang, and Trevor Darrell. Do convnets learn correspondence?, 2014. 8
 - [29] Aravindh Mahendran and Andrea Vedaldi. Understanding deep image representations by inverting them. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5188–5196, 2015. 8
 - [30] Meng Meng, Tianzhu Zhang, Qi Tian, Yongdong Zhang, and Feng Wu. Foreground activation maps for weakly supervised object localization. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3365–3375, 2021. 15
 - [31] Lucas Osco, José Junior, Ana Paula Ramos, Lucio Jorge, Sarah Narges Fatholahi, Jonathan Silva, Edson Matsubara, Hemerson Pistori, Wesley Gonçalves, and Jonathan Li. A review on deep learning in uav remote sensing, 01 2021. 5
 - [32] Dim P. Papadopoulos, Alasdair D. F. Clarke, Frank Keller, and Vittorio Ferrari. Training object class detectors from eye tracking data. In David Fleet, Tomas Pajdla, Bernt Schiele, and Tinne Tuytelaars, editors, *Computer Vision – ECCV 2014*, pages 361–376, Cham, 2014. Springer International Publishing. 11
 - [33] Ravindra Parmar. Detection and Segmentation through ConvNets. <https://medium.com/data-science/detection-and-segmentation-through-convnets-47aa42de27ea>. [Accessed 12-06-2026]. 9
 - [34] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos, 2024. 1
 - [35] Adrian Rosebrock. Intersection over union (iou) for object detection. PyImageSearch, 11 2016. Licensed under CC BY-SA 4.0. Wikimedia Commons: <https://commons.wikimedia.org/w/index.php?curid=57718559>. 21
 - [36] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3):211–252, 2015. 9
 - [37] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. *CoRR*, abs/1312.6034, 2013. 8
 - [38] Hao Su, J. Deng, and L. Fei-Fei. Crowdsourcing annotations for visual object detection. pages 40–46, 01 2012. 11
 - [39] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie. Caltech-ucsd birds-200-2011. Technical report, 2011. 10

- [40] Peter Welinder, Steve Branson, Takeshi Mita, Catherine Wah, Florian Schroff, Serge Belongie, and Pietro Perona. Caltech-ucsd birds 200. 09 2010. 26
- [41] Pingyu Wu, Wei Zhai, and Yang Cao. Background activation suppression for weakly supervised object localization. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14228–14237, 2022. 9, 15
- [42] Jilan Xu, Junlin Hou, Yuejie Zhang, Rui Feng, Rui-Wei Zhao, Tao Zhang, Xuequan Lu, and Shang Gao. Cream: Weakly supervised object localization via class re-activation mapping. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9427–9436, 2022. 9
- [43] Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks, 2013. 7, 8
- [44] Wei Zhai, Pingyu Wu, Kai Zhu, Yang Cao, Feng Wu, and Zheng-Jun Zha. Background activation suppression for weakly supervised object localization and semantic segmentation. *International Journal of Computer Vision*, 132(3):750–775, Mar 2024. 17
- [45] Chen-Lin Zhang, Yun-Hao Cao, and Jianxin Wu. Rethinking the route towards weakly supervised object localization. pages 13457–13466, 06 2020. 9
- [46] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Aude Oliva, and Antonio Torralba. Learning deep features for discriminative localization. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2921–2929, 2016. 9
- [47] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Aude Oliva, and Antonio Torralba. Object detectors emerge in deep scene cnns. 12 2014. 7